

비전 Edge AI**무선 액추에이터 제어**

- STEP 00 시스템 개요
- STEP 01 모니터·출력 구성
- STEP 02 준비물
- STEP 03 분류 모델 만들기
- STEP 04 비전 노드 펌웨어
- STEP 05 코어측 모니터 UI
- STEP 06 제어 노드 (OLED)
- STEP 07 게이지 다이얼 제작
- STEP 08 통합 테스트
- STEP 09 트러블슈팅·확장

공주고등학교

SW·AI 융합교육 · 2026-06

EDGE AI × BLE × ACTUATOR

비전 Edge AI 프로젝트 — 학생용 매뉴얼

카메라 보드가 기기 안에서 스스로(온디바이스) 물체를 알아봅니다. 무거운 영상은 **USB 선(유선)**으로 PC 화면에 보여 주고, 가벼운 판단 결과(단 3바이트)는 블루투스(**BLE 무선**)로 제어 보드에 보내 서보 바늘을 움직입니다. 우리가 만드는 것은 "AI가 알아본 물체를 바늘이 가리키는 비전 게이지"입니다.

Arduino Nicla Vision

Arduino Nano ESP32

Edge Impulse

BLE GATT

USB 영상 스트림

SG90 Servo

Streamlit

STEP 00

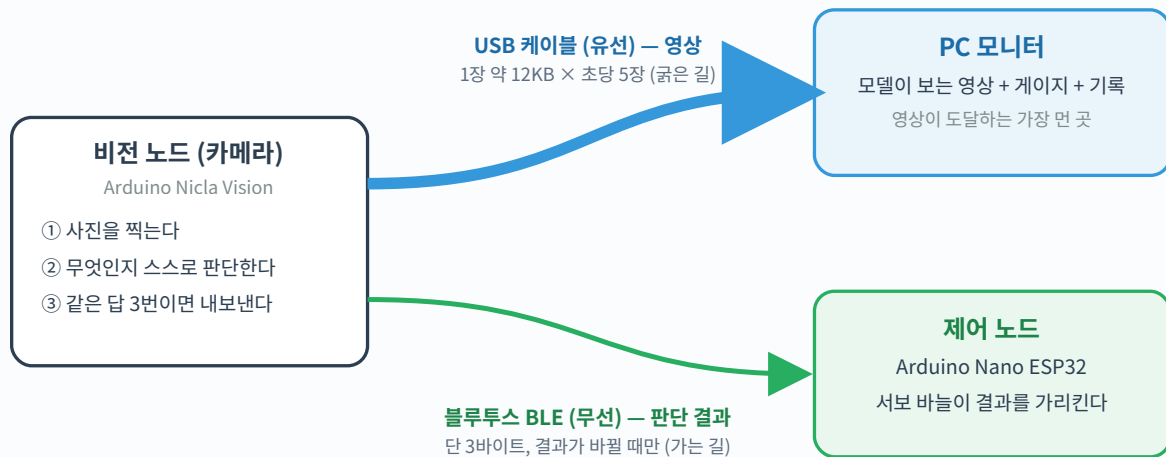
시스템 개요 — 영상은 유선으로, 판단은 무선으로

이 시스템의 설계 원칙은 "**전송 매체를 데이터의 무게에 맞춘다**"입니다. 용량이 큰 영상은 USB 선(유선)으로 보내고, 몇 바이트면 충분한 판단 결과만 무선(BLE)으로 보냅니다. 덕분에 무선 구간은 끝까지 가볍고 안정적입니다.

먼저 알아둘 용어 10가지

아래 용어만 알면 이 매뉴얼 전체를 읽을 수 있습니다. 읽다가 모르는 단어가 나오면 이 표로 돌아오세요.

용어	쉬운 설명
엣지 AI (온디바이스 AI)	인터넷 서버로 보내지 않고, 기기 안에서 AI가 스스로 판단하는 방식. 이 프로젝트의 핵심.
추론	학습을 마친 AI가 새 사진을 보고 "이건 컵"이라고 답을 내는 일.
분류 모델 / 라벨	정해진 보기 중 하나를 고르는 AI / 그 보기에 붙인 이름표(병, 컵, 가위 ...).
펌웨어 (스케치)	보드 안에 넣어 두고 돌리는 프로그램. 아두이노에서는 '스케치'라고 부릅니다.
업로드	컴퓨터에서 작성한 프로그램을 USB로 보드에 옮겨 넣는 것.
BLE (블루투스 저전력)	작은 데이터를 전기를 아끼며 보내는 근거리 무선. 무선 이어폰의 블루투스와의 형제.
바이트	데이터 크기의 기본 단위. 한글 한 글자가 2~3바이트, 사진 한 장은 수만 바이트.
서보 모터 (액추에이터)	신호를 받으면 정해진 각도로 도는 작은 모터. '움직이는 장치'를 통틀어 액추에이터라고 합니다.
GND (공통 접지)	전기의 기준 0V 선. 서로 다른 장치가 같은 기준선을 공유해야 신호를 올바르게 읽습니다.
안정화 필터	AI의 답이 흔들릴 때 바로 믿지 않고, 같은 답이 3번 연속 나올 때만 확정하는 규칙.



무거운 영상은 굵은 파란 길(유선)로, 가벼운 판단은 가는 초록 길(무선)로 — 길의 굵기가 곧 데이터의 무게입니다

왜 이런 구조인가?

- **길과 짐의 짝 맞추기** — 블루투스의 실제 전송 속도(초당 수십 KB)로는 영상을 보낼 수 없고, 그럴 필요도 없습니다. 영상은 유선으로, 결론은 무선으로 — 각 길에 잘 어울리는 짐만 맡깁니다.
- **역할 분리** — 비전 노드는 "보고 판단", 제어 노드는 "받아서 행동", PC는 "사람에게 보여주기". 실제 IoT의 표준 구조(센서 노드 ↔ 액추에이터 노드 ↔ 모니터링)를 그대로 체험합니다.
- **전송량의 통찰** — 무선으로는 영상이 아니라 **판단 결과 단 3바이트**만 이동합니다. "엣지에서 처리하고 결론만 보낸다"는 엣지 AI의 핵심 철학이 보내는 데이터의 크기에 그대로 드러납니다.

핵심 개념 ① — 3바이트 무선 프로토콜

BLE로 흐르는 데이터는 `[class_id, confidence, seq]` 단 3바이트입니다. `class_id`는 인식된 물체 번호(0~4), `confidence`는 확신도(0~100%), `seq`는 패킷 순번. 이 단순함이 BLE의 낮은 대역폭과 무선 공존 환경에서도 시스템을 안정적으로 만듭니다.

핵심 개념 ② — 줄 단위 시리얼 프로토콜

USB로 흐르는 데이터는 줄 단위 텍스트입니다: `IMG:<base64 프레임> 줄과 RES:{"cls":2,"conf":91} 줄`. 각 줄 앞에 `IMG;`, `RES:` 같은 머리말을 붙여 구분하므로 PC 쪽에서 읽기 쉽습니다. base64는 사진을 글자들로 바꿔 한 줄로 보내는 방식입니다. 영상은 모델 입력 그대로인 64×64 — "AI는 이 작은 그림으로 판단한다"를 보여주는 것 자체가 교육 포인트입니다.

핵심 개념 ③ — 두 보드, 두 BLE 라이브러리, 하나의 규격

Nicla Vision(mbed 코어)은 [ArduinoBLE](#) , Nano ESP32(esp32 코어)는 [BLEDevice.h](#) 를 씁니다. 코드 문법은 전혀 다르지만 무선 규격(BLE GATT)이 국제 표준이라 서로 완벽히 통신합니다. "라이브러리는 방언, 전파는 표준어" — 이질적 조합이 동작하는 것이 표준의 힘을 보여줍니다.

STEP 01

모니터 구성 — PC 화면으로 결과 보기

결과를 보는 화면은 **PC 모니터(코어측 모니터)** 하나입니다. 비전 노드의 USB 선을 PC에 꽂으면 모델이 보는 영상과 인식 결과 게이지가 화면에 나타납니다. 서보(물리 게이지)는 제어 노드에 직결돼 있어 **PC가 없어도 항상 움직입니다.**

기본

코어측 PC 모니터 — 영상 + 게이지 + 이력 ★ 개발·수업의 중심

비전 노드의 USB 시리얼을 PC가 직접 읽어 Streamlit으로 표시 — 유일하게 라이브 영상이 보이는 경로

- 모델이 보는 64×64 영상을 실시간 표시 (약 5fps — 추론 주기와 동기)
- 인식 결과 게이지 + 확신도 + 시간별 이력 로그까지 **가장 풍부한 UI**
- 무선 부담 0 추가 — 영상은 유선이므로 BLE는 그대로 한가함
- 켜고 끄기 = USB 연결 여부. 안 꽂으면 시스템은 무선 액추에이터만으로 자율 동작

사용 요령

USB를 꽂으면 화면이 켜지고, 뺄면 시스템은 무선 + 서보만으로 계속 동작합니다. 이 화면은 "모델이 지금 무엇을 보고 있는가"를 확인하는 창이라, 사진을 더 찍어야 할지·왜 오인식이 났는지 점검할 때 특히 유용합니다.

준비
물

구분	품목	비고
비전 노드	Arduino Nicla Vision	GC2145 2MP 카메라·Wi-Fi/ BLE 내장
제어 노드	Arduino Nano ESP32 + Grove Shield for Arduino Nano	⚠ 실드 전원 스위치 3V3 고정 (5V 금지)
액추에이터	SG90 마이크로 서보	180° 회전형 (360° 연속회전형 아님!)
서보 전원	LM2596 FND 전압표시 강압 컨버터 + 9~12V 어댑터	⚠ 무부하 5.0V 설정 후 연결 — STEP 06
배선 소품	Grove-to-male 점퍼 케이블, Grove 4핀 케이블, 헤더핀 (모듈 납땜용), 소형 일자 드라이버	드라이버는 트리머(W103) 조정용
공작 재료	두꺼운 종이/포맥스, 서보 혼, 양면테이프	게이지 다이얼판 제작용
소프트웨어	Arduino IDE 2.x + esp32 보드 패키지 + Arduino Mbed OS Nicla Boards	보드 매니저에서 각각 설치
라이브러리	ESP32Servo (Nano용) · ArduinoBLE (Nicla용)	라이브러리 매니저
클라우드 계정	Edge Impulse (무료)	Nicla Vision 공식 지원 보드
코어측 모니터 (기본)	PC + Python (pyserial, streamlit, plotly, numpy)	pip install pyserial streamlit plotly numpy
옵션① 추가	없음	폰/태블릿 1대면 충분
옵션② 추가	공유기 SSID/비밀번호, AP isolation 해제 확인	학교망은 관리자 확인 권장

인식 대상 정하기

모두이 자유롭게 선택 — 교실에서 구하기 쉽고 모양이 서로 뚜렷이 다른 물체 **2~5종**을 고르세요. 예: 물병·가위·마커펜·컵·안경·키보드·책·연필꽃이 등 무엇이든 좋습니다. 모양이 비슷한 물체(펜 vs 연필, 휴대폰 vs 지갑)는 피하는 것이 정확도에 유리합니다. unknown 클래스는 따로 학습시키지 않아도 됩니다 — 펌웨어의 임계값 필터(0.70 미만 → "모름")가 자동으로 처리합니다.

라벨 이름은 짧은 영문 단어로 지으세요 (예: `cup` , `pen` , `book` , `bottle` , `marker`). OLED 화면에 표시되므로 8자 이내가 보기 좋습니다. 길어도 화면에서 자동으로 글씨가 줄어들지만, 짧을수록 크고 선명하게 보입니다.

STEP 03

분류 모델 만들기

담당: Edge Impulse Studio (클라우드) — 학습 연산은 보드가 아니라 클라우드에서 일어납니다

WHAT — 이 단계가 하는 일

물체 사진 수백 장을 모아 클라우드에 올리고 라벨을 붙여 신경망을 학습시킵니다. 완성된 모델을 Nicla Vision용 Arduino 라이브러리 형태로 내려받습니다.

WHY — 왜 이렇게 하는가

학습에는 GPU급 연산이 필요해 보드에서는 불가능합니다. "학습은 클라우드, 추론은 엣지"로 나누면, 우리가 받는 건 학습이 끝난 모델의 추론 코드뿐이라 작은 보드에서도 돌아갑니다.

3-0. 사전 준비 — Nicla Vision에 Edge Impulse 펌웨어 굽기 (한 번만)

공장에서 막 꺼낸 Nicla Vision은 Edge Impulse 사이트와 직접 대화하지 못합니다. 보드에 **Edge Impulse용 펌웨어(전용 프로그램)**를 한 번 구워 두어야 사이트에서 "Connect a device"가 됩니다. 모뎀 한 번만 하면 되는 작업입니다.

1. **펌웨어 받기** — Edge Impulse 문서 페이지(docs.edgeimpulse.com/docs/edge-ai-hardware/mcu/arduino-nicla-vision)의 "Update the firmware" 섹션에서 zip 파일을 받아 압축을 풉니다. 폴더 안에 `firmware.bin` 과 `flash_windows.bat` (맥은 `flash_mac.command`)이 들어 있습니다.
2. **arduino-cli 설치** — flash 스크립트가 내부적으로 이 도구를 씁니다. 명령 프롬프트(Windows) 또는 터미널을 열고 한 줄:

```
winget install ArduinoSA.CLI
```

설치가 끝나면 **열려 있던 모든 명령 창을 닫고 새로 엽니다** (환경 변수가 새 창에만 적용됨). 새 창에서 `arduino-cli version` 을 쳤을 때 버전 번호가 나오면 성공.

3. **보드를 부트로더 모드로 진입** — Nicla Vision의 USB를 PC에 꽂은 상태에서, **리셋 버튼을 빠르게 두 번 클릭** (딸깍-딸깍, 마우스 더블클릭 정도). 성공 신호는 **녹색 LED가 천천히 호흡**(약 1초 주기로 밝아졌다 어두워졌다)하는 모습입니다. 한 번에 안 되면 간격을 살짝 바꿔 5~10번 시도하세요.
4. **플래시 실행** — 호흡 LED를 확인한 뒤 **5초 안에**, 압축 푼 폴더에서 `flash_windows.bat` 을 실행. 처음 실행 시 Mbed 코어를 자동 설치하느라 몇 분 걸릴 수 있습니다. 마지막에 "Flashed your Arduino Nicla Vision development board" 메시지가 나오면 완료.
5. **확인** — 보드가 자동 재부팅된 뒤, Edge Impulse Studio의 **Data acquisition** 탭에서 "Connect a device"를 누르면 이제 연결됩니다.

자주 겪는 문제

- ① **"arduino-cli not found"**: 2번 설치 후 명령 창을 새로 안 열어서 발생. 모든 창을 닫고 새로 여세요.
- ② **"No connected board found"**: 호흡 LED를 확인하지 않은 상태에서 실행. 더블탭부터 다시.
- ③ **"Failed to connect to device" (Studio 화면)**: 펌웨어를 안 굽고 바로 Studio에 연결을 시도한 경우. 1~5번 처음부터.

3-1. 프로젝트 생성과 데이터 수집

1. edgeimpulse.com 로그인 → **Create new project** (예: `vision-actuator`)
2. **Data acquisition** 탭 → 우측 USB 아이콘 → "Connect a device" → Nicla Vision이 WebUSB로 잡힘
3. **Sensor: Camera (64×64)** 선택. 96 이상은 USB 데이터가 커서 끊김이 잦습니다.
4. 각 클래스마다 **Label**(학습용 정답) 칸을 먼저 설정하고 → 물체를 화면에 두고 **Start sampling**
5. 클래스당 **60~80장**, 각도·거리·배경·조명을 다양하게. unknown 클래스를 두지 않을 경우 펌웨어의 임계값 필터(0.70)가 "모름" 처리합니다.
6. 업로드 시 학습용 80% / 테스트용 20% 자동 분할 확인.

실제 시연할 거리·배경·조명과 비슷하게 촬영하세요. "책상 위 30cm"에서 시연할 거면 학습 사진도 그렇게. 학습-시연 환경의 차이(도메인 차이)가 정확도 하락의 가장 흔한 원인입니다.

WebUSB 수집 안정성 — 차시 운영 가이드

WebUSB는 가끔 끊깁니다. 아래 5가지를 지키면 끊김이 크게 줄어듭니다.

- ① 해상도 64×64 고정 — 96 이상은 한 프레임이 너무 커서 USB 버퍼·메모리 단편화가 누적됩니다.
- ② 클래스당 한 번에 60장 말고 20장씩 3회로 끊어 찍기 — 사이사이 보드가 회복할 시간을 줍니다.
- ③ Chrome 브라우저, 다른 탭 닫기 — WebUSB는 브라우저 CPU 점유에 민감합니다.
- ④ 끊겼을 때 응급처치 30초: 페이지 새로고침 → "Connect a device" 다시 → 안 되면 USB 뽑았다 끼우기 → 그래도 안 되면 보드 리셋 버튼 한 번 + USB 재연결.
- ⑤ 특정 모듈만 계속 끊기면 케이블 교체. 충전 전용 케이블이 섞여 있는 경우가 종종 있습니다.

3-2. 임펄스 설계와 학습

1. **Create impulse**: 좌측 빨간 박스(Image data) — Image width/height 기본값(예: 64×64)으로 두고, Resize mode는 **Fit shortest axis** 그대로 유지. ※ 데이터 수집 해상도와 임펄스 입력 크기는 일치시키는 게 좋습니다.
2. **Add a processing block** → Image 선택 → Add
3. **Add a learning block** → Transfer Learning (Images) 선택 → Add. **Classification(일반)**이나 **Object Detection(FOMO)**을 같이 추가하지 마세요 — Transfer Learning 하나만으로 정확도가 가장 잘 나옵니다.
4. 오른쪽 초록색 Output features 박스가 우리 클래스 수(3)와 같으면 정상 → **Save Impulse**
5. 좌측 메뉴 **Image** → Color depth = RGB 확인 → Save parameters → **Generate features** (3~5분). Feature explorer에서 클래스별 점이 색깔로 잘 갈리면 학습이 잘 될 신호입니다.
6. 좌측 메뉴 **Transfer learning** → 기본 모델(MobileNetV2 ...) 유지, Training cycles 20, Learning rate 0.0005, Training processor **GPU**(EI 클라우드 자원) → **Save & train**
7. **Confusion matrix**로 클래스별 정확도 확인 — 85% 이상 합격. 특정 클래스만 0%에 가까우면 사진 보강 후 재학습
8. (선택) **Live classification**에서 카메라를 실제로 비춰 봅니다. 시연 환경에서 빈 책상이나 무관한 물체의 확신도가 50% 미만으로 떨어지면, unknown 클래스 없이 가도 됩니다 — 펌웨어의 임계값 필터(0.70)가 알아서 "모름" 처리합니다.

unknown 클래스가 꼭 필요한가?

모델 학습 단계에서 unknown 클래스를 따로 두지 않아도, 펌웨어에서 "가장 높은 확신도가 0.70 미만이면 모름 처리"로 간단히 거를 수 있습니다. 둘 다 가능하지만, 후자가 모델이 더 가볍고 두 진짜 클래스의 정확도가 더 잘 나옵니다.

라벨링 함정 — Label과 Sample name은 다릅니다

Data acquisition 화면에서 **Label** 입력 칸을 먼저 찍을 물체에 맞게 설정한 뒤 촬영하세요. Label은 학습에 쓰이는 정답이고, Sample name(파일 이름)은 사람이 알아보기 위한 메모입니다. 잘못 붙은 라벨로 학습하면 "안경처럼 생긴 것 = pen"처럼 엉뚱한 학습이 됩니다.

3-3. 배포 (모델 내려받기)

1. **Deployment** 탭 → Deployment target: **Arduino library**
2. Inference engine: **EON™ Compiler** (기본 선택). 같은 모델을 더 작게 빌드해 줍니다 — RAM 약 19%, 플래시 약 21% 절감. Nicla의 RAM 여유가 빠듯하므로 끄지 마세요.
3. Model optimizations: **Quantized (int8)** 선택 (대개 기본). 추론 속도는 약 4배 빨라지고, 정확도 손실은 거의 없습니다.
4. 하단 **Build** 클릭 → 1~2분 후 zip 자동 다운로드
5. Arduino IDE에서 **스케치** → **라이브러리 포함** → **.ZIP 라이브러리 추가**로 zip 설치
6. **파일** → **예제** → **ei-(프로젝트명)-arduino** → **nicla_vision** → **nicla_vision_camera** 예제가 보이면 성공 — 이것이 STEP 04 펌웨어의 카메라 글루 부품으로 쓰입니다.

EON Compiler가 뭔가?

Edge Impulse가 만든 "모델 코드 압축기"입니다. 일반적으로 임베디드 보드는 TensorFlow Lite Micro라는 만능 인터프리터를 함께 올리는데, 이 도구는 우리 모델이 실제로 쓰는 연산만 골라 전용 C++ 코드로 변환해 줍니다. 정확도 손실 없이 RAM·플래시 사용량을 줄여 작은 보드에서 더 큰 모델을 돌릴 수 있게 합니다.

STEP 04

비전 노드 펌웨어 — 단계별로 만들고 확인하기

담당: **Arduino Nicla Vision** — 카메라로 찍고, 모델에 넣어 답을 내고, 같은 답이 3번 연속이면 "확정"이라고 외친다

WHAT — 이 단계가 하는 일

EI에서 받은 라이브러리를 써서 카메라 프레임을 모델에 넣고 결과를 받습니다. 결과가 흔들리지 않게 "같은 답 3번 연속" 필터를 거친 뒤, USB로는 PC 모니터에, 무선(BLE)으로는 제어 노드에 보냅니다.

WHY — 한 번에 다 만들지 않는 이유

한 번에 만들면 어디서 무엇이 안 되는지 모릅니다. ① 코어 추론 → ② USB 스트리밍 + UI → ③ BLE 송신의 레이어를 단계별로 추가하며, 매 단계 끝에 눈으로 확인합니다. 어떤 단계가 안 되면 직전 단계만 의심하면 됩니다.

4-1. 보드 설정 — 한 번만

- 보드 매니저: **Arduino Mbed OS Nicla Boards** 설치 → 보드 **Arduino Nicla Vision** 선택
- 라이브러리: STEP 03에서 받은 **EI zip**을 **.ZIP 라이브러리** 추가로 설치 (3-A에서 구운 펌웨어와는 별개 — 학습된 모델 라이브러리입니다)
- 업로드 실패/포트 사라짐 시: **리셋 버튼 빠르게 2회** → 부트로더 모드(녹색 LED 호흡) → 재업로드

4-A. 코어 테스트 — 추론 + 안정화 필터 (USB·BLE 없음)

가장 단순한 형태로 시작합니다. 카메라 → 추론 → 시리얼 모니터에 텍스트만 출력. 이 단계에서 검증할 것: 카메라 캡처가 되는가, 모델이 로드되는가, 안정화 필터가 의도대로 작동하는가, 추론 시간이 합리적인가.

1. 아래 코드를 복사하여 새 스케치(예: `vision_node_step1_test.ino`)에 붙여넣고 업로드 (컴파일 1~2분)

코드 `vision_node_step1_test.ino` — 코어 동작 테스트

```
/* =====
 * Vision Gauge — STEP 4-A · 코어 테스트 펌웨어
 *
 * 이 파일에서 검증하는 것:
 * 1) 카메라(GC2145)가 정상 캡처되는가
 * 2) Edge Impulse 모델이 보드에서 추론을 돌리는가
 * 3) 안정화 필터(같은 답 3회 + conf >= 0.70)가 의도대로 작동하는가
 *
 * 이 파일에 없는 것 (다음 단계에서 추가):
 * - USB 시리얼 영상 스트리밍 (STEP 4-B 에서 추가)
 * - BLE 무선 송신 (STEP 4-C 에서 추가)
 * - OLED 화면 출력 (STEP 06, 제어 노드 측)
 *
 * 결과 확인: Arduino IDE 시리얼 모니터 (115200 baud)
 * 매 추론마다 RES: <label> (<conf>) <ms>ms
 * 안정화 통과 시 >>> STABLE: <label> (<conf>) <<<
 * ===== */

#include <object_clf_inferencing.h> // EI Studio에서 받은 라이브러리 헤더
// 학습된 모델 가중치, 라벨, run_classifier() 함수가 모두 들어 있음
#include "edge-impulse-sdk/dsp/image/image.hpp" // RGB565 -> RGB888 변환, 리사이즈 함수
#include "camera.h" // Arduino Nicla Camera 클래스
#include "gc2145.h" // GC2145 센서 드라이버
#include <ea_malloc.h> // mbed 환경의 정렬 메모리 할당
```

```

/* =====
* 1. 설정값 (여기만 조정하면 동작이 바뀜)
* ===== */
#define CONF_THRESHOLD 0.70f // 이 확신도 이상일 때만 결과로 인정 (0.0 ~ 1.0)
// 올리면 더 엄격 (놓치는 결과 늘고, 오인식 줄어듦)
// 낮추면 더 관대 (반응 빠르고, 오인식 늘어남)
#define STABLE_REQUIRED 3 // 같은 답이 N번 연속이면 "확정"으로 판정
// 바늘 떨림 / OLED 깜빡임을 방지하는 핵심 장치
#define LOOP_INTERVAL_MS 100 // 추론 시도 간격(ms) - 실제 추론은 ~32ms 걸리므로
// 대략 100ms 주기 = 초당 10번 시도 (5fps급)

/* =====
* 2. 카메라 글루 코드 (EI 공식 예제에서 무수정 이식)
* -----
* 이 영역은 GC2145 센서를 EI의 추론 파이프라인에 연결하는 "어댑터".
* 학생 차시에서 직접 손댈 일은 거의 없고, 그대로 두고 동작.
*
* 동작 흐름:
* 카메라 -> 320x240 RGB565 프레임 -> RGB888로 변환
* -> 모델 입력 크기(64x64)로 리사이즈/크롭
* -> EI 추론기에 픽셀 데이터 공급
* ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320 // 카메라 출력 폭 (센서 고정)
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240 // 카메라 출력 높이
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE 2 // RGB565는 픽셀당 2바이트
// 32바이트 정렬 매크로 - DMA 전송 시 정렬되지 않으면 성능 저하 또는 오동작
#define ALIGN_PTR(p,a) ((p & (a-1)) ? (((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)

GC2145 galaxyCore; // GC2145 센서 드라이버 인스턴스
Camera cam(galaxyCore); // Arduino Camera API 래퍼
FrameBuffer fb; // 프레임 버퍼 객체

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL; // RGB888 변환/리사이즈 결과 버퍼
static uint8_t *ei_camera_frame_mem; // 원본 RGB565 프레임 메모리
static uint8_t *ei_camera_frame_buffer; // 32바이트 정렬된 포인터

typedef struct { size_t width; size_t height; } resize_t;

/* RGB565(2바이트/픽셀) -> RGB888(3바이트/픽셀) 변환
* 카메라는 메모리 절약을 위해 RGB565를 출력하지만, 모델은 RGB888을 입력으로 받음 */
bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8; // R (상위 5비트)
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3; // G (중간 6비트)
        *dst++ = (lb & 0x1F) << 3; // B (하위 5비트)
    }
    return true;
}

/* 출력 해상도에 맞는 중간 리사이즈 크기 결정.
* EI 라이브러리가 단계적으로 줄여나가는 방식을 사용 */
int calc_resize(uint32_t out_w, uint32_t out_h,
                uint32_t *col, uint32_t *row, bool *do_resize) {
    const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
    *col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
    *row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
    *do_resize = false;
    for (size_t i = 0; i < 5; i++) {
        if (out_w <= list[i].width && out_h <= list[i].height) {
            *col = list[i].width; *row = list[i].height; *do_resize = true; break;
        }
    }
    return 0;
}

```

```

/* 카메라 초기화 - setup()에서 한 번만 호출 */
bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;

    // 원본 RGB565 프레임 메모리 할당 (320 * 240 * 2 = 153,600 byte + 정렬 여유)
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;

    // 32바이트 정렬된 위치로 포인터 조정 (DMA 효율)
    ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
    fb.setBuffer(ei_camera_frame_buffer);
    is_initialised = true;
    return true;
}

/* 한 프레임 캡처 + RGB 변환 + 리사이즈
 * 호출 후 ei_camera_capture_out 에 모델 입력 크기로 준비된 RGB888 픽셀이 들어 있음 */
bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;

    // RGB888 출력 버퍼 할당 (320 * 240 * 3 = 230,400 byte + 정렬)
    // 함수 종료 시 ea_free로 반환 - 단편화 방지
    ei_camera_capture_out = (uint8_t*)ea_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);

    // 1) 카메라에서 RGB565 프레임 한 장 받기 (100ms 타임아웃)
    if (cam.grabFrame(fb, 100) != 0) return false;

    // 2) RGB888로 변환
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize()))
        return false;

    // 3) 모델 입력 크기로 리사이즈 + 중앙 크롭
    uint32_t rc, rr; bool do_resize;
    calc_resize(w, h, &rc, &rr, &do_resize);
    if (do_resize) {
        ei::image::processing::crop_and_interpolate_rgb888(
            ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
            ei_camera_capture_out, rc, rr);
    }
    return true;
}

/* EI 추론기가 픽셀을 가져갈 때 호출하는 콜백.
 * RGB888 3바이트를 32비트 정수 하나로 패킹해 모델에 전달 */
static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
 * 3. 안정화 필터 상태 (전역 변수)
 * =====
 * 안정화 필터의 핵심 아이디어:
 * 카메라가 흔들리거나 빛이 변하면 모델이 잠깐 다른 답을 내기도 함.
 * -> "확신도 0.70 이상" + "같은 답이 3번 연속"을 모두 통과한 결과만

```

```

*      "STABLE = 확정"으로 인정. 그 외에는 무시.
* 이 필터 덕분에 OLED 표시가 깜빡이지 않고, 서보 바늘이 떨지 않음.
* ===== */
int lastClass    = -1;    // 직전 추론에서 가장 확신도 높았던 클래스
int stableCount  = 0;    // 같은 답이 몇 번 연속됐는지 카운트
int confirmedCls = -1;    // 최근 "STABLE"로 확정된 클래스 (변화 시점 감지용)

/* =====
* 4. setup() - 부팅 시 한 번만 실행
* ===== */
void setup() {
    Serial.begin(115200);
    // USB 시리얼 연결을 5초간 기다림 (PC가 안 켜져 있어도 진행)
    while (!Serial && millis() < 5000) ;

    Serial.println();
    Serial.println("=====");
    Serial.println("Vision Gauge - STEP 4-A : Nicla 코어 테스트");
    Serial.println("=====");

    // [중요] Nicla Vision의 M4 코어 RAM 영역을 추가 할당 영역으로 등록.
    // 기본 M7 RAM(512KB)만으로는 EI TFLite arena가 부족해 부팅 직후 리셋됨.
    // 이 한 줄로 M4 영역에서 288KB를 추가 확보 - Arduino 공식 권장.
    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) {
        Serial.println("FATAL: 카메라 초기화 실패");
        while(1); // 무한 루프 (FATAL 표시 후 정지)
    }
    Serial.println("OK: 카메라 초기화 완료");

    // 학습된 모델 정보를 시리얼에 출력 - 학생이 자기 라벨이 맞는지 확인 가능
    Serial.print("모델 입력: ");
    Serial.print(EI_CLASSIFIER_INPUT_WIDTH); Serial.print("x");
    Serial.println(EI_CLASSIFIER_INPUT_HEIGHT);
    Serial.print("클래스 수: "); Serial.println(EI_CLASSIFIER_LABEL_COUNT);
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        Serial.print("  "); Serial.print(i); Serial.print(" ");
        // 라벨은 EI Studio에서 자동으로 알파벳순으로 정렬됨
        // STEP 06의 LABELS[] 배열 순서가 이 순서와 같아야 함
        Serial.println(ei_classifier_inferencing_categories[i]);
    }
    Serial.print("필터: conf>="); Serial.print(CONF_THRESHOLD);
    Serial.print(", stable="); Serial.println(STABLE_REQUIRED);
    Serial.println("추론 루프 시작...");
    Serial.println();
}

/* =====
* 5. loop() - 계속 반복 실행
* =====
* 매 LOOP_INTERVAL_MS(=100ms)마다:
* 1) 카메라 캡처 2) EI 추론 3) 가장 확신도 높은 클래스 선택
* 4) 시리얼에 결과 출력 5) 안정화 필터 적용
* ===== */
void loop() {
    // 100ms 간격 유지 - 매 루프 즉시 추론하면 보드가 과열되고 USB도 막힘
    static uint32_t lastTick = 0;
    if (millis() - lastTick < LOOP_INTERVAL_MS) return;
    lastTick = millis();

    uint32_t t0 = millis(); // 추론 시간 측정 시작

    // 1) 카메라 캡처
    if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
        ei_free(ei_camera_capture_out);
        return;
    }

```

```

}

// 2) 추론 - EI가 제공하는 signal_t 구조를 통해 픽셀 데이터를 모델에 전달
ei::signal_t signal;
signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
signal.get_data      = &ei_camera_get_data;    // 위에서 정의한 콜백

ei_impulse_result_t result = {0};
EI_IMPULSE_ERROR err = run_classifier(&signal, &result, false);
if (err != EI_IMPULSE_OK) {
    Serial.print("ERR: run_classifier "); Serial.println(err);
    ea_free(ei_camera_capture_out);
    return;
}

// 3) argmax - 모델은 모든 클래스의 확률을 반환. 그중 가장 큰 것 하나만 선택
float maxConf = 0;
int maxIdx = -1;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > maxConf) {
        maxConf = result.classification[i].value;
        maxIdx = i;
    }
}

uint32_t totalMs = millis() - t0;    // 캡처+추론 총 소요시간

// 4) 시리얼에 매 추론 결과 출력 (사람이 읽을 수 있도록 라벨명까지)
Serial.print("RES: ");
Serial.print(ei_classifier_inferencing_categories[maxIdx]);
Serial.print(" ("); Serial.print(maxConf, 2); Serial.print(") ");
Serial.print(totalMs); Serial.println("ms");

// 5) 안정화 필터 - "같은 답 N회 + conf >= 임계값" 통과 시에만 STABLE 확정
if (maxConf >= CONF_THRESHOLD) {
    if (maxIdx == lastClass) {
        stableCount++;                // 같은 답 -> 카운트 증가
    } else {
        stableCount = 1;              // 다른 답 -> 카운트 리셋
        lastClass = maxIdx;
    }

    // STABLE 통과 + "직전 확정과 다른 새 답"일 때만 STABLE 줄 출력
    // (같은 답이 계속 STABLE로 들어와 줄이 도배되는 걸 방지)
    if (stableCount >= STABLE_REQUIRED && maxIdx != confirmedCls) {
        confirmedCls = maxIdx;
        Serial.print(">>> STABLE: ");
        Serial.print(ei_classifier_inferencing_categories[confirmedCls]);
        Serial.print(" ("); Serial.print(maxConf, 2); Serial.println(") <<<");
    }
} else {
    // 확신도가 임계값 미만 - "모름" 상태로 간주, 카운터 리셋
    // (이게 곧 unknown 클래스 없이도 "모름"을 처리하는 핵심 메커니즘)
    stableCount = 0;
    lastClass = -1;
}

ea_free(ei_camera_capture_out);    // 캡처 버퍼 반환 - 단편화 방지
}

/* =====
* 6. 안전 검증 - EI 라이브러리가 카메라 입력 모델인지 확인
* ===== */
#if !defined(EI_CLASSIFIER_SENSOR) || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "이 펌웨어는 카메라 입력 모델 전용입니다 (EI 프로젝트 설정 확인)"
#endif

```

2. 시리얼 모니터(115200 baud) 열기 → 다음과 같이 흐르면 성공 (아래 라벨은 예시 — 여러분이 학습한 라벨로 다르게 표시됩니다):

```
=====
Vision Gauge — Nicla Core Test
=====
OK: camera initialized
Model input: 64x64
Classes (3):
  [0] glasses
  [1] keyboard
  [2] unkown
Filter: conf>=0.70, stable=3
Starting inference loop...

RES: unkown (0.45) 32ms
RES: glasses (0.88) 32ms    ← 학습한 물체A를 카메라 앞에 둠
RES: glasses (0.91) 31ms
RES: glasses (0.89) 32ms
>>> STABLE: glasses (0.89) <<<    ← 3회 연속 → 확정
```

합격 체크리스트

① 클래스 3개가 알파벳순으로 출력 ② 추론 시간 30~150ms (Quantized int8 + EON이면 ~32ms, 둘 중 하나라도 빠지면 ~140ms) ③ 안경/키보드를 들이대면 conf 0.85 이상 ④ 물체 변경 시 약 0.6초 후 >>> STABLE 이 한 번 뜸 ⑤ 5~10분 방치해도 리셋·멈춤 없음

업로드 직후 빨간 경고 — 무시

업로드 마지막에 `Warning: Invalid DFU suffix signature` 가 빨간 글씨로 떠도 무시하세요. 바로 아래 `File downloaded successfully` 가 보이면 펌웨어는 정상 전송된 것입니다. dfu-util(펌웨어 굽기 도구)이 미래 호환성을 미리 알리는 경고일 뿐, 동작에는 영향 없습니다.

4-B. USB 스트리밍 + 모니터 UI 레이어 추가

4-A에서 텍스트만 봤다면, 이제 PC 화면에서 영상과 함께 봅니다. 펌웨어에 줄 단위 프로토콜을 추가하고, PC에는 Streamlit으로 모니터 UI를 띄웁니다.

4-B-1. 펌웨어 갱신

1. 아래 코드로 교체 업로드 — 추가 내용은 줄 단위 프로토콜 3종입니다:

```

/* =====
 * Vision Gauge - STEP 4-B : USB 스트리밍 + 모니터 UI 레이어 추가
 *
 * step1과의 차이 (이 파일에서 새로 추가된 것):
 *   [+ STEP 4-B] base64 인코더 함수 (serialPrintBase64)
 *   [+ STEP 4-B] setup() 끝에 LBL: 라벨 목록 송신
 *   [+ STEP 4-B] loop()의 USB 스트리밍 블록 (RES: / IMG: 줄)
 *   [+ STEP 4-B] STABLE 시 사람용 로그 형식 변경 (# STABLE: ...)
 *   [+ STEP 4-B] USB 미연결 시 송신 생략 가드 (if (Serial))
 *
 * step1과 동일한 것 (변경 없음):
 *   - 카메라 글루, EI 추론, argmax, 안정화 필터 로직
 *
 * PC쪽 monitor.py가 받는 줄 단위 프로토콜:
 *   LBL:<label1>,<label2>,...   부팅 시 1회 (학습한 라벨 목록)
 *   RES:<idx>,<conf>,<t_ms>     매 추론 (가공 전 raw 결과)
 *   IMG:<base64 64x64 RGB888> 3번에 1번 (모델이 본 영상)
 *   # STABLE: <label> <conf> seq=<n>   안정화 확정 시
 * ===== */

#include <object_clf_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
#include "camera.h"
#include "gc2145.h"
#include <ea_malloc.h>

/* =====
 * 1. 설정값 (step1과 동일 + IMG 송신 빈도만 추가)
 * ===== */
#define CONF_THRESHOLD      0.70f
#define STABLE_REQUIRED    3
#define LOOP_INTERVAL_MS   100
#define IMG_EVERY_N        3      // [+ STEP 4-B] 3번 추론마다 1프레임 전송 (영상 부드러움)
                                   // 매 추론마다 영상을 보내면 USB 대역폭 초과
                                   // IMG는 약 12KB 텍스트 - 5fps 정도가 적절

/* =====
 * 2. 카메라 글루 (step1과 100% 동일 - 변경 없음)
 * ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS  320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS  240
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE    2
#define ALIGN_PTR(p,a) ((p & (a-1)) ? (((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)

GC2145 galaxyCore;
Camera cam(galaxyCore);
FrameBuffer fb;

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL;
static uint8_t *ei_camera_frame_mem;
static uint8_t *ei_camera_frame_buffer;

typedef struct { size_t width; size_t height; } resize_t;

bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8;
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3;
        *dst++ = (lb & 0x1F) << 3;
    }
}

```



```

    return true;
}

int calc_resize(uint32_t out_w, uint32_t out_h,
               uint32_t *col, uint32_t *row, bool *do_resize) {
    const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
    *col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
    *row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
    *do_resize = false;
    for (size_t i = 0; i < 5; i++) {
        if (out_w <= list[i].width && out_h <= list[i].height) {
            *col = list[i].width; *row = list[i].height; *do_resize = true; break;
        }
    }
    return 0;
}

bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;
    ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
    fb.setBuffer(ei_camera_frame_buffer);
    is_initialised = true;
    return true;
}

bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;
    ei_camera_capture_out = (uint8_t*)ea_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);
    if (cam.grabFrame(fb, 100) != 0) return false;
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize())) return false;
    uint32_t rc, rr; bool do_resize;
    calc_resize(w, h, &rc, &rr, &do_resize);
    if (do_resize) {
        ei::image::processing::crop_and_interpolate_rgb888(
            ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
            ei_camera_capture_out, rc, rr);
    }
    return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
* [+ STEP 4-B 추가] 3. base64 인코더
* -----
* 왜 필요한가:
* 카메라 영상은 RGB888 바이너리 데이터 (값에 0x0A=개행, 0x00=NULL 등이 섞임).
* 바이너리를 그대로 시리얼로 보내면 줄 단위 파싱이 깨짐.
* -> base64로 변환하면 알파벳+숫자+/+ 만 사용해 텍스트 라인으로 안전 전송.
* 비용:
* 3바이트 -> 4글자로 늘어남 (33% 오버헤드). 64x64x3=12,288바이트 -> 16,384글자.

```

```

/* ===== */
static const char b64chars[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

// 인코드 결과를 그대로 Serial로 출력해보냄 (메모리 절약 - 큰 버퍼 안 잡음)
void serialPrintBase64(const uint8_t *data, size_t len) {
    for (size_t i = 0; i < len; i += 3) {
        // 3바이트씩 묶어 24비트 정수 v로 합침
        uint32_t v = data[i] << 16;
        if (i + 1 < len) v |= data[i + 1] << 8;
        if (i + 2 < len) v |= data[i + 2];
        // 24비트를 6비트씩 4조각으로 잘라 base64 문자 인덱스로 사용
        Serial.print(b64chars[(v >> 18) & 0x3F]);
        Serial.print(b64chars[(v >> 12) & 0x3F]);
        Serial.print(i + 1 < len ? b64chars[(v >> 6) & 0x3F] : '='); // 패딩
        Serial.print(i + 2 < len ? b64chars[v & 0x3F] : '=');
    }
}

/* =====
* 4. 안정화 필터 상태 (step1과 동일 + 송신용 변수 추가)
* ===== */
int    lastClass    = -1;
int    stableCount  = 0;
int    confirmedCls = -1;
uint8_t seqNum      = 0; // [+ STEP 4-B] STABLE 시 sequence 번호 (PC측 ID)
uint32_t loopCount  = 0; // [+ STEP 4-B] IMG 송신 빈도 카운터

/* [+ STEP 4-B] 라벨 목록 송신 함수
* monitor.py가 이 LBL: 줄을 받아 "인덱스 -> 라벨명" 매핑을 만들.
* 부팅 시 1회 + loop에서 주기적으로 재전송 (monitor가 나중에 연결돼도 라벨 수신) */
void sendLabels() {
    Serial.print("LBL:");
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        if (i) Serial.print(",");
        Serial.print(ei_classifier_inferencing_categories[i]);
    }
    Serial.println();
}

/* =====
* 5. setup() - step1 + 라벨 송신 추가
* ===== */
void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 5000) ;

    // [+ STEP 4-B] 시리얼 출력 형식 변경
    // step1: 사람 친화적인 긴 메시지
    // step2: monitor.py가 파싱하기 쉽도록 # 주석 + LBL/RES/IMG 줄 위주
    Serial.println("# Vision Gauge v2 - USB 스트리밍");

    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) { Serial.println("# FATAL: 카메라"); while(1); }

    // [+ STEP 4-B] 라벨 목록 송신 (sendLabels 함수로 분리 - loop에서 주기 재전송)
    sendLabels();
    Serial.println("# ready");
}

/* =====
* 6. loop() - step1 추론 + [+ STEP 4-B] USB 스트리밍 블록
* ===== */
void loop() {
    static uint32_t lastTick = 0;
    if (millis() - lastTick < LOOP_INTERVAL_MS) return;

```

```

lastTick = millis();

// [+ STEP 4-B] 라벨 목록 주기적 재전송 (5초마다)
// monitor.py를 나중에 실행해도 라벨을 받을 수 있도록 - "#2" 대신 이름 표시
static uint32_t lastLbl = 0;
if (Serial && millis() - lastLbl > 5000) {
    lastLbl = millis();
    sendLabels();
}

uint32_t t0 = millis();
if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
    ea_free(ei_camera_capture_out); return;
}

// 추론 (step1과 동일)
ei::signal_t signal;
signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
signal.get_data      = &ei_camera_get_data;
ei_impulse_result_t result = {0};
if (run_classifier(&signal, &result, false) != EI_IMPULSE_OK) {
    ea_free(ei_camera_capture_out); return;
}

// argmax (step1과 동일)
float maxConf = 0; int maxIdx = -1;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > maxConf) {
        maxConf = result.classification[i].value; maxIdx = i;
    }
}
uint32_t totalMs = millis() - t0;

/* ----- [+ STEP 4-B 추가] USB 송신 블록 -----
 * if (Serial) 가드:
 *   USB가 연결 안 됐을 때는 송신을 통째로 건너뛴.
 *   안 그러면 시리얼 버퍼가 꽉 차서 다음 추론이 늦어질 수 있음.
 *   (시연 시 PC 없이 보드만 동작시킬 때 중요)
 */
if (Serial) {
    // RES: 줄 - 매 추론마다 (가공 전 raw 결과)
    Serial.print("RES:");
    Serial.print(maxIdx); Serial.print(",");
    Serial.print(maxConf, 3); Serial.print(",");
    Serial.println(totalMs);

    // IMG: 줄 - 3번에 1번 (모델이 본 영상을 base64 텍스트로)
    if (loopCount % IMG_EVERY_N == 0) {
        Serial.print("IMG:");
        size_t pixBytes = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT * 3;
        serialPrintBase64(ei_camera_capture_out, pixBytes);
        Serial.println();
    }
}
loopCount++;

// 안정화 필터 (step1과 거의 동일, STABLE 출력 형식만 변경)
if (maxConf >= CONF_THRESHOLD) {
    if (maxIdx == lastClass) stableCount++;
    else { stableCount = 1; lastClass = maxIdx; }

    if (stableCount >= STABLE_REQUIRED && maxIdx != confirmedCls) {
        confirmedCls = maxIdx;
        seqNum++;
        // [+ STEP 4-B] 사람용 로그 형식 - monitor.py가 "# STABLE:"으로 파싱
        // PC쪽 모니터에서 "확정 결과" 박스를 갱신하는 트리거
        Serial.print("# STABLE: ");

```

```

Serial.print(ei_classifier_inferencing_categories[confirmedCls]);
Serial.print(" "); Serial.print(maxConf, 2);
Serial.print(" seq="); Serial.println(seqNum);
// [+ STEP 4-C 예정] 여기서 BLE notify로 3바이트 송신 (step3에서 추가)
}
} else {
    stableCount = 0; lastClass = -1;
}

ea_free(ei_camera_capture_out);
}

#if !defined(EI_CLASSIFIER_SENSOR) || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "이 펌웨어는 카메라 입력 모델 전용입니다"
#endif

```

줄	의미	빈도
LBL:<라벨1>,<라벨2>,...	학습한 라벨 목록 — PC가 인덱스→이름을 자동 동기화. 예: LBL:bottle,marker,pen	부팅 1회
RES:<idx>,<conf>,<t_ms>	매 추론 결과 (안정화 전 raw)	매 추론
IMG:<base64 64×64 RGB888>	모델이 본 영상 — 약 12KB/장	3번에 1번

2. 업로드 후 시리얼 모니터(115200)로 잠깐 확인 — 위 세 종류 줄이 흐르면 성공. **확인 후 시리얼 모니터는 꼭 닫으세요** — PC 모니터 UI가 같은 포트를 열어야 합니다.

4-B-2. PC 모니터 UI — uv 가상환경 + Streamlit

1. PowerShell에서 uv 설치 (한 번만, 이미 깔려 있으면 건너뛰기):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

설치 후 모든 PowerShell 창을 닫고 새로 엽니다.

2. 프로젝트 폴더 생성 + 패키지 설치:

```
cd $HOME
mkdir vision_gauge
cd vision_gauge
uv init
uv add streamlit pyserial numpy pillow
```

3. 아래 코드를 `vision_gauge/monitor.py` 로 저장

코드 monitor.py — Streamlit 모니터 UI

```
"""
=====
Vision Gauge - 코어측 모니터 UI (STEP 4-B / STEP 05)
=====
Nicla Vision의 USB 시리얼을 읽어 PC 화면에 표시하는 Streamlit 앱.

Nicla Vision이 보내는 줄 단위 프로토콜:
LBL:<라벨1>,<라벨2>,...   부팅 1회 (학습한 라벨 목록)
RES:<idx>,<conf>,<t_ms>   매 추론 (raw 결과)
IMG:<base64 64x64 RGB888> 3번에 1번 (모델이 본 영상)
# STABLE: <라벨> <conf> seq=<n>   안정화 확정 시
# ...                           그 외 주석 (디버깅용, UI에는 미사용)

화면 구성:
좌측 - 64x64 영상 (5배 확대)
우측 - 실시간 인식 카드 (클래스별 색상, 큰 폰트, 퍼센트)
하단 - 최근 20개 이력 표

실행:
uv run streamlit run monitor.py
=====
"""

import base64
import io
import threading
import time
from collections import deque
from dataclasses import dataclass, field

import numpy as np
import serial
import serial.tools.list_ports
import streamlit as st
from PIL import Image

# ----- 상수 -----
IMG_W, IMG_H = 64, 64      # 모델 입력 해상도 (펌웨어와 일치해야 함)
BAUD = 115200              # 시리얼 통신 속도 (펌웨어 Serial.begin과 같아야)
POLL_MS = 150              # UI 갱신 주기 (150ms = 초당 약 6.7회, 영상 부드러움)

# 클래스별 색상 팔레트 - 다크 배경에서 서로 명확히 대비되는 테마 컬러맵
# (Tableau 10 계열을 다크 테마에 맞게 밝기 보정 - 인접 색이 확연히 구분됨)
CLASS_COLORS = [
    "#4FC3F7", # 0 하늘색
    "#FF7043", # 1 주황
    "#66BB6A", # 2 초록
    "#BA68C8", # 3 보라
```

```

"#FFCA28", # 4 노랑
"#EC407A", # 5 핑크
"#26C6DA", # 6 청록
"#9CCC65", # 7 연두
]

def class_color(idx: int) -> str:
    """클래스 인덱스 -> 색상 (개수 초과 시 순환)"""
    return CLASS_COLORS[idx % len(CLASS_COLORS)]

def semicircle_gauge(pct: int, color: str) -> str:
    """반원형(180도) 게이지 SVG 생성.
    pct: 0~100 확신도, color: 채움 색상.
    바늘이 좌(0%)에서 우(100%)로 회전, 중심축 하단에 Confidence 표기."""
    import math
    # 반원: 왼쪽(180도)에서 오른쪽(0도)으로. pct에 따라 바늘 각도 계산
    # 180도(왼쪽 끝) ~ 0도(오른쪽 끝)
    angle_deg = 180 - (pct / 100) * 180
    angle_rad = math.radians(angle_deg)
    cx, cy, r = 130, 130, 100 # 중심, 반지름
    # 바늘 끝점
    nx = cx + r * 0.82 * math.cos(angle_rad)
    ny = cy - r * 0.82 * math.sin(angle_rad)

    # 호(arc) 경로 - 배경(회색 전체 반원) + 채움(색상, pct만큼)
    def arc_point(p):
        a = math.radians(180 - (p / 100) * 180)
        return (cx + r * math.cos(a), cy - r * math.sin(a))
    bg_start = arc_point(0)
    bg_end = arc_point(100)
    fill_end = arc_point(pct)
    large = 0 # 반원이므로 항상 0
    sweep_bg = 1
    # 채움 호: 0%(왼쪽)에서 pct까지
    fill_path = (f"M {bg_start[0]:.1f} {bg_start[1]:.1f} "
                f"A {r} {r} 0 0 1 {fill_end[0]:.1f} {fill_end[1]:.1f}")
    bg_path = (f"M {bg_start[0]:.1f} {bg_start[1]:.1f} "
               f"A {r} {r} 0 0 1 {bg_end[0]:.1f} {bg_end[1]:.1f}")

    return f"""
<svg viewBox="0 0 260 170" style="width:100%;max-width:340px;">
  <!-- 배경 호 (회색) -->
  <path d="{bg_path}" fill="none" stroke="#1e293b" stroke-width="20"
        stroke-linecap="round"/>
  <!-- 채움 호 (색상) -->
  <path d="{fill_path}" fill="none" stroke="{color}" stroke-width="20"
        stroke-linecap="round"/>
  <!-- 바늘 -->
  <line x1="{cx}" y1="{cy}" x2="{nx:.1f}" y2="{ny:.1f}"
        stroke="#f1f5f9" stroke-width="4" stroke-linecap="round"/>
  <!-- 중심축 -->
  <circle cx="{cx}" cy="{cy}" r="9" fill="#f1f5f9"/>
  <circle cx="{cx}" cy="{cy}" r="4" fill="{color}"/>
  <!-- 퍼센트 숫자 (호 안쪽 상단, 바늘과 겹치지 않게) -->
  <text x="{cx}" y="78" text-anchor="middle" fill="#f1f5f9"
        font-size="36" font-weight="800">{pct}</text>
  <!-- Confidence 라벨 (중심축 하단) -->
  <text x="{cx}" y="158" text-anchor="middle" fill="#94a3b8"
        font-size="15" font-weight="600" letter-spacing="0.1em">Confidence</text>
</svg>"""

# =====
# 1. 백그라운드 스레드용 공유 상태
# -----
# 시리얼 읽기는 블로킹 작업이므로 별도 스레드에서 처리.

```

```
# Streamlit 메인 스레드는 이 State를 읽어 화면만 그림.
# =====
@dataclass
class State:
    labels: list = field(default_factory=list) # LBL:로 받은 라벨 목록
    last_img: np.ndarray = None # 마지막 IMG: 디코드 결과
    last_res: dict = None # 최근 RES: {"label", "conf", "t_ms"}
    history: deque = field(default_factory=lambda: deque(maxlen=20)) # 이력
    stable: dict = None # 최근 STABLE 결과
    connected: bool = False # 시리얼 연결 상태
    error: str = "" # 에러 메시지
    ble_tx: str = "idle" # BLE 송신 상태: idle/sent/waiting
    ble_tx_time: float = 0.0 # 마지막 BLE 송신 시각

# =====
# 2. 시리얼 읽기 백그라운드 루프
# -----
# - PySerial로 한 줄씩 읽기
# - 머리말(LBL:/RES:/IMG:/# STABLE:)로 분기해 State 갱신
# - stop_flag로 종료 신호 받음
# =====
def reader_loop(port: str, state: State, stop_flag: list):
    try:
        ser = serial.Serial(port, BAUD, timeout=1)
        state.connected = True
        state.error = ""
    except Exception as e:
        state.error = f"포트 열기 실패: {e}"
        return

    try:
        while not stop_flag[0]:
            try:
                line = ser.readline().decode("utf-8", errors="ignore").strip()
            except Exception:
                continue
            if not line:
                continue

            # ---- LBL: 라벨 목록 (부팅 1회) ----
            if line.startswith("LBL:"):
                state.labels = line[4:].split(",")

            # ---- RES: 매 추론 결과 ----
            elif line.startswith("RES:"):
                try:
                    idx_str, conf_str, ms_str = line[4:].split(",")
                    idx = int(idx_str)
                    conf = float(conf_str)
                    t_ms = int(ms_str)
                    label = state.labels[idx] if 0 <= idx < len(state.labels) else f"#{idx}"
                    state.last_res = {"label": label, "conf": conf, "t_ms": t_ms, "idx": idx}
                    state.history.append({
                        "시각": time.strftime("%H:%M:%S", time.localtime()),
                        "라벨": label,
                        "확신도": f"{int(round(conf*100))}%",
                    })
                except ValueError:
                    pass # 파싱 실패는 무시 (잡음 줄)

            # ---- IMG: 영상 base64 ----
            elif line.startswith("IMG:"):
                try:
                    raw = base64.b64decode(line[4:])
                    # 길이 검증 - 줄이 잘리지 않았는지 확인
                    if len(raw) == IMG_W * IMG_H * 3:

```

```

        arr = np.frombuffer(raw, dtype=np.uint8).reshape(IMG_H, IMG_W, 3)
        state.last_img = arr
    except Exception:
        pass

# ---- # STABLE: 안정화 확정 ----
# 예: "# STABLE: bottle 0.89 seq=3"
elif line.startswith("# STABLE:"):
    try:
        parts = line[len("# STABLE:").:].strip().split()
        label = parts[0]
        conf = float(parts[1])
        # 라벨 -> 인덱스 (색상 매핑용)
        idx = state.labels.index(label) if label in state.labels else 0
        state.stable = {"label": label, "conf": conf, "idx": idx,
                        "time": time.strftime("%H:%M:%S")}
    except Exception:
        pass

# ---- BLE 송신 상태 (step3가 # TX: 줄로 출력) ----
# "... (BLE central에 notify 전송)" -> 송신 성공
# "... (BLE 미연결 ...)" -> 수신자 없음
elif "BLE central" in line and "notify" in line:
    state.ble_tx = "sent"
    state.ble_tx_time = time.time()
elif "BLE 미연결" in line:
    state.ble_tx = "waiting"
    state.ble_tx_time = time.time()

# 그 외 "# ..." 주석 줄은 무시 (디버그용)
finally:
    ser.close()
    state.connected = False

# =====
# 3. Streamlit UI - 메인 스레드
# =====
st.set_page_config(page_title="Vision Gauge Monitor", layout="wide")
st.title(" Vision Gauge Monitor")
st.caption("Nicla Vision의 USB 시리얼 출력을 실시간으로 확인")

# ---- 사이드바: 포트 선택 + 연결 토글 ----
with st.sidebar:
    st.subheader("연결")
    ports = [p.device for p in serial.tools.list_ports.comports()]
    if not ports:
        st.warning("시리얼 포트를 찾을 수 없습니다. 보드가 USB로 연결되었는지 확인하세요.")
        st.stop()

    selected_port = st.selectbox("포트", ports)

    if "thread" not in st.session_state:
        st.session_state.thread = None
        st.session_state.state = State()
        st.session_state.stop_flag = [False]

    if st.button("연결" if not st.session_state.state.connected else "연결 끊기"):
        if st.session_state.thread is None or not st.session_state.thread.is_alive():
            # 시작
            st.session_state.state = State()
            st.session_state.stop_flag = [False]
            t = threading.Thread(
                target=reader_loop,
                args=(selected_port, st.session_state.state, st.session_state.stop_flag),
                daemon=True,
            )

```



```

        t.start()
        st.session_state.thread = t
    else:
        # 종료
        st.session_state.stop_flag[0] = True
        st.session_state.thread = None

# 상태 표시
state: State = st.session_state.state
if state.connected:
    st.success(f"연결됨: {selected_port}")
elif state.error:
    st.error(state.error)
else:
    st.info("대기 중")

# ---- BLE 송신 상태 인디케이터 ----
# 비전 노드가 STABLE 시 BLE notify를 보내는지 USB 로그로 추적
st.markdown("***BLE 송신***")
ble_fresh = (time.time() - state.ble_tx_time) < 3.0 # 3초 내 활동
if state.ble_tx == "sent" and ble_fresh:
    st.success("    송신 중 (제어 노드 수신)")
elif state.ble_tx == "waiting" and ble_fresh:
    st.warning("⚠ 수신자 없음 (제어 노드 미연결)")
else:
    st.caption("📡 대기 — STABLE 확정 시 송신")

if state.labels:
    st.markdown("***라벨***")
    for i, lab in enumerate(state.labels):
        st.write(f"`[{i}]` {lab}")

# ---- 메인 영역: 영상 + 결과 (둘 다 정사각형, 1:1 배치) ----
col_img, col_res = st.columns([1, 1])

with col_img:
    st.subheader("모델이 보는 영상")
    img_area = st.empty()

with col_res:
    st.subheader("실시간 인식 결과")
    res_area = st.empty()

st.subheader("최근 20개 이력")
hist_area = st.empty()

# ---- POLL_MS마다 화면 갱신 (67회 x 150ms 약 10초 후 rerun) ----
# Streamlit은 자동 새로고침이 없으므로 명시적으로 st.rerun() 호출
POLL_MS_total = POLL_MS / 1000
for _ in range(67):
    state: State = st.session_state.state

    # 영상 - 정사각형(aspect-ratio 1:1). base64로 HTML img에 삽입
    if state.last_img is not None:
        img = Image.fromarray(state.last_img).resize((IMG_W * 5, IMG_H * 5),
                                                       Image.NEAREST)

        buf = io.BytesIO()
        img.save(buf, format="PNG")
        b64 = base64.b64encode(buf.getvalue()).decode()
        img_area.markdown(f"""
<div style="aspect-ratio:1/1;border-radius:18px;overflow:hidden;margin-top:8px;
            background:#000;display:flex;align-items:center;justify-content:center;">

</div>
""", unsafe_allow_html=True)

```

```

        else:
            img_area.markdown("""
<div style="aspect-ratio:1/1;border-radius:18px;margin-top:8px;background:#1e293b;
        display:flex;align-items:center;justify-content:center;
        color:#94a3b8;font-size:16px;">영상 대기 중...</div>
""", unsafe_allow_html=True)

        # ---- 실시간 인식 (영상과 같은 높이의 큰 카드, 클래스별 색상) ----
        if state.last_res:
            r = state.last_res
            # 라벨을 아직 못 받아 인덱스(#N)로 표시되는 경우 - 동기화 안내
            if r["label"].startswith("#"):
                res_area.markdown("""
<div style="background:#1e293b;border:2px dashed #475569;border-radius:18px;
        box-sizing:border-box;padding:40px 32px;margin-top:8px;text-align:center;
        display:flex;flex-direction:column;justify-content:center;align-items:center;
        aspect-ratio:1/1;">
<div style="font-size:22px;color:#94a3b8;">라벨 동기화 중...</div>
<div style="font-size:15px;color:#64748b;margin-top:10px;">
    비전 노드에서 라벨 목록을 받는 중입니다 (몇 초 소요)</div>
</div>
""", unsafe_allow_html=True)
            else:
                color = class_color(r["idx"])
                pct = int(round(r["conf"] * 100))
                # 절충안(실시간 송신): STABLE 개념 대신 확신도로 상태 구분
                if pct >= 70:
                    check = "● 안정적 인식"
                elif pct >= 50:
                    check = "● 인식 중"
                else:
                    check = "○ 탐색 중"

                gauge = semicircle_gauge(pct, color)
                res_area.markdown(f"""
<div style="background: linear-gradient(135deg,{color}26,{color}0a);
        border:2px solid {color};border-radius:18px;box-sizing:border-box;
        padding:28px 32px;margin-top:8px;aspect-ratio:1/1;
        display:flex;flex-direction:column;align-items:center;
        justify-content:center;text-align:center;">
<div style="font-size:16px;color:{color};letter-spacing:0.08em;
        font-weight:600;margin-bottom:10px;">{check}</div>
<div style="font-size:60px;font-weight:800;color:{color};
        line-height:1.05;margin-bottom:8px;
        word-break:break-all;text-shadow:0 2px 12px {color}40;">{r['label']}</div>
        {gauge}
</div>
""", unsafe_allow_html=True)
            else:
                res_area.markdown("""
<div style="background:#1e293b;border:2px dashed #475569;border-radius:18px;
        box-sizing:border-box;padding:40px 32px;margin-top:8px;text-align:center;
        display:flex;flex-direction:column;justify-content:center;align-items:center;
        aspect-ratio:1/1;">
<div style="font-size:22px;color:#94a3b8;">추론 결과 대기 중...</div>
</div>
""", unsafe_allow_html=True)

        # 이력 표
        if state.history:
            rows = list(state.history)[::-1]
            hist_area.dataframe(rows, use_container_width=True, hide_index=True)
        else:
            hist_area.empty()

        time.sleep(POLL_MS_total)

```

```
st.rerun() # 약 10초마다 페이지 자동 새로고침 - Streamlit 특성상 필요
```

4. 실행:

```
uv run streamlit run monitor.py
```

브라우저가 자동으로 열립니다. 안 열리면 콘솔의 `http://localhost:8501` 주소를 직접 열기.

매번 명령 치기 번거롭다면 — 배치파일(.bat)로 더블클릭 실행

터미널에서 폴더로 이동해 `uv run streamlit run monitor.py` 를 매번 입력하는 대신, 배치파일을 만들어 더블클릭 한 번으로 실행할 수 있습니다.

1. 메모장을 열고 아래 4줄을 입력합니다:

```
@echo off
cd /d "%~dp0"
uv run streamlit run monitor.py
pause
```

2. 다른 이름으로 저장 → 파일 형식을 "모든 파일(*.*)"로 바꾸고, 인코딩 UTF-8, 이름은 `대시보드_실행.bat` (반드시 `.bat` 로 끝나게).

3. 이 `.bat` 파일을 `monitor.py`와 같은 폴더에 두고 더블클릭하면 끝.

핵심 한 줄 — `cd /d "%~dp0"` 의 `%~dp0` 는 "이 배치파일이 있는 폴더"를 자동으로 가리킵니다. 덕분에 폴더를 옮기거나 다른 PC에 복사해도 경로 수정 없이 그대로 동작합니다. (`@echo off` 는 명령어 표시를 끄고, `pause` 는 오류 시 창이 바로 닫히지 않게 합니다.)

바탕화면에서 바로 실행 — .bat 우클릭 → "추가 옵션 표시"(Windows 11) → "보내기" → "바탕 화면(바로 가기 만들기)". 단, 원본 .bat은 monitor.py 폴더에 그대로 뒀야 합니다(바로가기는 원본 위치를 기준으로 동작).

5. 사이드바에서 포트(예: COM5) 선택 → 연결. 약 1초 후:

- 상태가 **연결됨**으로 바뀌고, 클래스 목록이 사이드바에 표시
- 왼쪽에 64×64 영상이 5배 확대되어 표시 (1초마다 갱신)
- 오른쪽 위에 실시간 인식 결과(클래스명, 확신도, 추론 시간)
- 오른쪽 아래에 안정화 확정 결과(초록 박스)

- 아래쪽에 최근 인식 이력 표

한 번에 하나만 — 시리얼 포트는 독점

같은 COM 포트는 한 프로그램만 점유할 수 있습니다. **Arduino IDE 시리얼 모니터 ↔ monitor.py**는 동시에 못 열고, **monitor.py 동작 중에는 펌웨어 재업로드도 불가**합니다. 재업로드 시엔 사이드바의 ■ 정지를 먼저 누르세요.

왜 uv 가상환경인가

PC의 시스템 파이썬에 직접 설치하면 다른 프로그램과 충돌할 수 있고, 학생 PC에 같은 환경을 옮기기도 번거롭습니다. uv는 ① 시스템 파이썬과 격리, ② 설치 속도 pip보다 10~100배 빠름, ③ `pyproject.toml` 한 파일로 환경 복제 — 학교 단위 운영에 가장 안전한 선택입니다.

4-C. BLE 송신 레이어 추가

4-B까지로 "PC와 함께 보는 엣지 AI 카메라"가 완성되었습니다. 이제 USB가 없어도 동작하는 무선 채널을 더합니다 — 최신 추론 결과를 3바이트로 묶어 200ms마다 실시간으로 BLE 송신. 다음 STEP 06의 OLED 제어 노드가 이 무선 신호를 받아 화면에 표시합니다.

4-C-1. 라이브러리 설치

- Arduino IDE 라이브러리 매니저 → **"ArduinoBLE"** 검색 → Arduino 공식 라이브러리 설치 (현재 v1.3.x)
- ⚠ 이 라이브러리는 Nicla Vision(mbed 보드)용. 제어 노드(ESP32)에서는 `BLEDevice.h` (다른 라이브러리)를 사용 — 같은 BLE라도 보드별 문법이 달라 호환되지 않습니다. 자세한 건 STEP 06.

4-C-2. 펌웨어 갱신

step2 코드에 BLE 초기화·송신 로직을 더한 step3을 업로드합니다. 변경점은 크게 4가지:

1. `#include <ArduinoBLE.h>` + 서비스/특성 객체 선언
2. `setup()` 에서 `BLE.begin()` → 서비스·특성 등록 → `BLE.advertise()`
3. 200ms마다 최신 추론 결과 3바이트 `[class_id, conf, seq]` 를 `resultChar.writeValue()` 로 실시간 송신
4. `loop()` 시작마다 `BLE.poll()` 호출 — **이게 빠지면 연결 직후 끊깁니다** (가장 흔한 함정)

코드 vision_node_step3_ble.ino — BLE 송신 레이어 추가

```
/* =====  
 * Vision Gauge - STEP 4-C : BLE 송신 레이어 추가  
 *  
 * step2와의 차이 (이 파일에서 새로 추가된 것):
```

```

*   [+ STEP 4-C] #include <ArduinoBLE.h>
*   [+ STEP 4-C] BLE 서비스/특성 객체 전역 선언 (결과 + 라벨)
*   [+ STEP 4-C] setup()에서 BLE.begin -> advertise (5단계)
*   [+ STEP 4-C] loop() 첫 줄 BLE.poll() (생략 시 끊김!)
*   [+ STEP 4-C] 200ms마다 최신 추론 결과를 3바이트 notify 송신 (실시간)
*
* step2와 동일한 것 (변경 없음):
*   - 카메라 글루, EI 추론, argmax
*   - USB 스트리밍 (LBL/RES/IMG)
*
* BLE 송신 방식 (절충안 - 실시간):
*   안정화 필터(STABLE) 없이 매 추론의 최신 결과를 200ms마다 송신.
*   확신도가 낮아도 즉시 반영되고, 200ms 쓰로틀로 OLED 깜빡임을 억제.
*
* BLE 송신 데이터:
*   3바이트 [class_id, conf*100, seq]
*   class_id : 클래스 인덱스 (0~N-1, 알파벳순)
*   conf*100 : 확신도 정수 (0~100, 0.85 -> 85)
*   seq      : 시퀀스 번호 (0~255 순환, 송신마다 증가)
*
* 검증 절차:
*   1) USB 시리얼로 RES/IMG/TX 줄이 흐르는지 (4-B + TX)
*   2) 폰 nRF Connect 앱으로 "VisionGauge" 검색/연결
*   3) 0x7e400002 특성 구독 후 물체 변경 시 3바이트 갱신 확인
*   ===== */

#include <object_clf_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
#include "camera.h"
#include "gc2145.h"
#include <ea_malloc.h>
#include <ArduinoBLE.h>          // [+ STEP 4-C] BLE Peripheral API

/* =====
* 1. 설정값
* ===== */
#define LOOP_INTERVAL_MS 100 // 추론 시도 간격(ms)
#define IMG_EVERY_N 3 // IMG 송신 빈도 (3번에 1번)
#define BLE_TX_INTERVAL_MS 200 // [절충안] BLE 송신 주기 - 200ms마다 최신 결과
// 짧으면 반응 빠르나 OLED 깜빡임/대역폭 ↑
// 길면 부드러우나 반응 느림. 200ms가 균형점

/* =====
* [+ STEP 4-C 추가] 2. BLE 서비스/특성 정의
* -----
* UUID는 임의의 128비트 값 - "이 서비스는 다른 BLE 기기와 충돌하지 않는다"
* 는 의미만 가짐. 0x7e40 prefix는 우리 프로젝트 고유 식별자로 사용.
* 제어 노드(ESP32)의 코드도 이 UUID와 정확히 일치해야 함.
* ===== */
#define BLE_DEVICE_NAME "VisionGauge" // nRF Connect 스캔에 표시될 이름
#define SERVICE_UUID "7e400001-b5a3-f393-e0a9-e50e24dcca9e"
#define CHAR_UUID "7e400002-b5a3-f393-e0a9-e50e24dcca9e"
#define LABEL_UUID "7e400003-b5a3-f393-e0a9-e50e24dcca9e" // [+ 라벨 특성]

BLEService visionService(SERVICE_UUID);
// 결과 특성 - 3바이트 [class_id, conf*100, seq], 자주 갱신
// BLERead : Central이 값을 읽을 수 있음
// BLENotify : 값이 바뀌면 Central에 자동 푸시 (구독 시)
BLECharacteristic resultChar(CHAR_UUID, BLERead | BLENotify, 3);

// [+ 라벨 특성] 라벨 목록 문자열 "label1,label2,unknown"
// 부팅 시 1회 기록, 제어 노드가 연결 직후 read해서 LABELS[] 채움.
// 최대 64바이트 (라벨이 길거나 많으면 늘릴 것)
BLECharacteristic labelChar(LABEL_UUID, BLERead, 64);

/* =====
* 3. 카메라 글루 (step1/step2와 100% 동일)

```

```

* ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE 2
#define ALIGN_PTR(p,a) ((p & (a-1)) ? (((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)

GC2145 galaxyCore;
Camera cam(galaxyCore);
FrameBuffer fb;

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL;
static uint8_t *ei_camera_frame_mem;
static uint8_t *ei_camera_frame_buffer;

typedef struct { size_t width; size_t height; } resize_t;

bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8;
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3;
        *dst++ = (lb & 0x1F) << 3;
    }
    return true;
}

int calc_resize(uint32_t out_w, uint32_t out_h,
                uint32_t *col, uint32_t *row, bool *do_resize) {
    const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
    *col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
    *row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
    *do_resize = false;
    for (size_t i = 0; i < 5; i++) {
        if (out_w <= list[i].width && out_h <= list[i].height) {
            *col = list[i].width; *row = list[i].height; *do_resize = true; break;
        }
    }
    return 0;
}

bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;
    ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
    fb.setBuffer(ei_camera_frame_buffer);
    is_initialised = true;
    return true;
}

bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;
    ei_camera_capture_out = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);
    if (cam.grabFrame(fb, 100) != 0) return false;
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize())) return false;
    uint32_t rc, rr; bool do_resize;
    calc_resize(w, h, &rc, &rr, &do_resize);
    if (do_resize) {
        ei::image::processing::crop_and_interpolate_rgb888(
            ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,

```

```

        ei_camera_capture_out, rc, rr);
    }
    return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
* 4. base64 인코더 (step2에서 추가, 그대로 유지)
* ===== */
static const char b64chars[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

void serialPrintBase64(const uint8_t *data, size_t len) {
    for (size_t i = 0; i < len; i += 3) {
        uint32_t v = data[i] << 16;
        if (i + 1 < len) v |= data[i + 1] << 8;
        if (i + 2 < len) v |= data[i + 2];
        Serial.print(b64chars[(v >> 18) & 0x3F]);
        Serial.print(b64chars[(v >> 12) & 0x3F]);
        Serial.print(i + 1 < len ? b64chars[(v >> 6) & 0x3F] : '=');
        Serial.print(i + 2 < len ? b64chars[(v >> 0) & 0x3F] : '=');
    }
}

/* =====
* 5. 송신 상태 변수
* ===== */
uint8_t seqNum = 0; // BLE 송신 시퀀스 번호 (0~255 순환)
uint32_t loopCount = 0; // IMG 송신 빈도 카운터

/* 라벨 목록 USB 송신 함수 (부팅 1회 + loop 주기 재전송)
* monitor.py가 나중에 연결돼도 라벨을 받아 "#2" 대신 이름 표시 */
void sendLabels() {
    Serial.print("LBL:");
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        if (i) Serial.print(",");
        Serial.print(ei_classifier_inferencing_categories[i]);
    }
    Serial.println();
}

/* =====
* 6. setup() - step2 + [+ STEP 4-C] BLE 초기화 5단계
* ===== */
void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 5000) ;

    Serial.println("# Vision Gauge v3 - USB + BLE");
    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) { Serial.println("# FATAL: 카메라"); while(1); }

    /* [+ STEP 4-C 추가] BLE 초기화 5단계 (순서 바꾸면 동작 안 함)
    * 1) BLE.begin() BLE 스택 기동
    * 2) setLocalName() 기기 이름 (스캔 시 표시)
    * 3) setAdvertisedService() 광고에 서비스 UUID 포함
    */

```

```

* 4) addCharacteristic + addService 특성을 서비스에 등록
* 5) advertise() 광고 시작 (Central이 발견 가능)
*/
if (!BLE.begin()) {
    Serial.println("# FATAL: BLE.begin failed");
    while(1);
}
BLE.setLocalName(BLE_DEVICE_NAME);
BLE.setAdvertisedService(visionService);
visionService.addCharacteristic(resultChar);
visionService.addCharacteristic(labelChar); // [+ 라벨 특성] 등록
BLE.addService(visionService);

// 초기값 [0, 0, 0] - 첫 연결 시 Central이 읽으면 이 값을 받음
uint8_t initVal[3] = {0, 0, 0};
resultChar.writeValue(initVal, 3);

// [+ 라벨 특성] 라벨 목록을 쉼표로 이어 문자열로 기록
// 예: "bottle,marker,unknown"
// 제어 노드(ESP32)가 연결 직후 이 값을 read해서 LABELS[]를 채움
String labelStr = "";
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (i) labelStr += ",";
    labelStr += ei_classifier_inferencing_categories[i];
}
labelChar.writeValue(labelStr.c_str());
Serial.print("# 라벨 특성 기록: "); Serial.println(labelStr);

BLE.advertise();
Serial.print("# BLE advertising as ");
Serial.print(BLE_DEVICE_NAME);
Serial.println("");

// 라벨 목록 송신 (sendLabels로 분리 - loop에서 주기 재전송)
sendLabels();
Serial.println("# ready");
}

/* =====
* 7. loop() - step2 + [+ STEP 4-C] BLE.poll() + BLE notify
* ===== */
void loop() {
    /* [+ STEP 4-C 추가] BLE 이벤트 처리 - 매 루프 반드시 호출!
    * 누락 시 증상:
    * - 연결은 되지만 1~2초 후 끊어짐 (Central은 PHY layer만 보고 alive 판단)
    * - 학생 차시 BLE 문제의 80%가 이 한 줄 누락
    * 위치는 loop() 첫 줄이 권장 - 추론 사이에 끼면 추론 시간이 길어짐
    */
    BLE.poll();

    static uint32_t lastTick = 0;
    if (millis() - lastTick < LOOP_INTERVAL_MS) return;
    lastTick = millis();

    // 라벨 목록 주기적 재전송 (5초마다) - monitor.py 나중 연결 대비
    static uint32_t lastLbl = 0;
    if (Serial && millis() - lastLbl > 5000) {
        lastLbl = millis();
        sendLabels();
    }

    uint32_t t0 = millis();
    if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
        ea_free(ei_camera_capture_out); return;
    }

    // 추론 (step1/step2와 동일)

```



```

ei::signal_t signal;
signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
signal.get_data      = &ei_camera_get_data;
ei_impulse_result_t result = {0};
if (run_classifier(&signal, &result, false) != EI_IMPULSE_OK) {
    ea_free(ei_camera_capture_out); return;
}

// argmax
float maxConf = 0; int maxIdx = -1;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > maxConf) {
        maxConf = result.classification[i].value; maxIdx = i;
    }
}
uint32_t totalMs = millis() - t0;

/* USB 송신 (step2와 동일) - BLE와 독립적으로 동작 */
if (Serial) {
    Serial.print("RES:");
    Serial.print(maxIdx); Serial.print(",");
    Serial.print(maxConf, 3); Serial.print(",");
    Serial.println(totalMs);

    if (loopCount % IMG_EVERY_N == 0) {
        Serial.print("IMG:");
        size_t pixBytes = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT * 3;
        serialPrintBase64(ei_camera_capture_out, pixBytes);
        Serial.println();
    }
}
loopCount++;

/* [+ STEP 4-C / 절충안] BLE 실시간 송신
 * STABLE 조건 없이 매 추론 결과를 200ms마다 최신값으로 송신.
 * - 즉각 반응 (확신도 낮아도 보임)
 * - 200ms 쓰로틀로 OLED 깜빡임 억제 (매 추론마다 보내면 너무 잦음)
 * monitor.py는 이 송신을 BLE 인디케이터에 반영. */
static uint32_t lastBleMs = 0;
if (millis() - lastBleMs >= BLE_TX_INTERVAL_MS) {
    lastBleMs = millis();
    seqNum++;

    // 3바이트 [class_id, conf*100, seq] - 가장 확신도 높은 클래스 그대로
    uint8_t bleBuf[3] = {
        (uint8_t)maxIdx,
        (uint8_t)(maxConf * 100), // 0.0~1.0 -> 0~100 정수
        seqNum
    };
    resultChar.writeValue(bleBuf, 3);

    // 사람용 로그 (monitor.py가 BLE 송신 인디케이터 트리거로 사용)
    Serial.print("# TX: ");
    Serial.print(ei_classifier_inferencing_categories[maxIdx]);
    Serial.print(" "); Serial.print(maxConf, 2);
    if (BLE.connected()) {
        Serial.println(" (BLE central에 notify 전송)");
    } else {
        Serial.println(" (BLE 미연결 - 수신자 없음)");
    }
}

ea_free(ei_camera_capture_out);
}

#ifdef EI_CLASSIFIER_SENSOR || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA

```

```
#error "이 펌웨어는 카메라 입력 모델 전용입니다"
#endif
```

4-C-3. 검증 — 폰 nRF Connect 앱

BLE 송신이 정말 나가는지 확인하려면 폰 BLE 디버거 앱이 가장 편합니다. 제어 노드(STEP 06) 만들기 전에 폰으로 먼저 확인하면 문제 격리가 쉽습니다.

1. 폰 앱스토어에서 **nRF Connect** 설치 (Nordic Semiconductor 제공, 무료)
2. 앱 실행 → SCAN → 목록에서 "**VisionGauge**" 찾아 CONNECT
3. 연결 후 "Unknown Service" 안의 **0x7e400002** 특성을 펼치고 구독 아이콘(↓↓ 또는 **Notify**) 탭
4. 비전 노드 카메라 앞에 학습한 물체를 들이대면 **Value 영역에 3바이트가 갱신**됩니다. 예: `0x01 0x5A 0x03` = 클래스 1번(라벨 두 번째), 확신도 90 (=0x5A), 시퀀스 3
5. 물체를 바꾸면 약 0.6초 후 새 3바이트가 도착

합격 체크리스트

- ① 보드 부팅 시 시리얼에 "BLE advertising as 'VisionGauge'" 출력
- ② nRF Connect 스캔에 "VisionGauge"가 보임
- ③ 연결 시 시리얼에 "STABLE" 줄과 "(BLE central에 notify 전송)" 줄이 함께 출력
- ④ 폰 화면에서 3바이트 값이 물체 변경 시마다 갱신
- ⑤ USB 시리얼 모니터링(monitor.py)도 BLE와 동시에 동작 — 두 채널이 독립

자주 만나는 문제

- ① "**BLE.begin failed**" — Nicla 보드 패키지가 오래된 경우. 보드 매니저에서 "Arduino Mbed OS Nicla Boards" 최신으로 업데이트.
- ② nRF Connect 스캔에 "**VisionGauge**"가 안 보임 — 같은 폰의 다른 BLE 앱(이전 세션)이 점유 중일 수 있음. 폰 블루투스 껐다 켜기.
- ③ 연결 직후 1~2초 만에 끊김 — `BLE.poll()` 누락. step3 코드의 loop() 첫 줄에 있는지 확인.
- ④ 3바이트 값이 안 갱신 (연결은 되지만) — 안정화 필터가 안 통과한 상태. 시리얼 모니터에서 STABLE 줄이 뜨는지 먼저 확인. 안 뜨면 confidence가 0.70 미만이라는 뜻.

STEP 05

모니터 UI의 동작 원리 — monitor.py 들여다보기

담당: PC (Python + Streamlit) — STEP 04-B에서 이미 실행했습니다. 이 STEP은 "어떻게 동작하는가"를 이해하는 것이 목적.

WHAT — monitor.py가 하는 일

USB 시리얼 포트를 백그라운드 스레드에서 읽다가 줄의 머리말로 분기합니다: **LBL:** 은 라벨 동기화, **RES:** 는 게이지·이력 갱신, **IMG:** 는 base64 디코딩 → 영상 표시.

WHY — 왜 한 포트에 다 받나

비전 노드가 한 줄씩 다른 머리말을 붙여 보내므로, PC는 포트 하나만 열어도 영상·결과·라벨이 동시에 들어옵니다. 시리얼 모니터처럼 단순하지만 시각화는 훨씬 풍부합니다.

5-1. 줄 단위 프로토콜 — 3종

줄	처리
LBL:<라벨1>,<라벨2>,...	쉼표로 split → 학습한 라벨 리스트 보관. 이후 RES 인덱스를 이름으로 변환할 때 사용 — 모델 라벨 순서가 바뀌어도 PC가 자동으로 따라옴 (예: <code>LBL:bottle,marker,pen</code>)
RES:0,0.894,32	<code>int(idx)</code> , <code>float(conf)</code> , <code>int(t_ms)</code> 파싱 → 실시간 인식 게이지 갱신 + 이력 큐에 추가
IMG:<base64>	base64 디코딩 → 12,288바이트($64 \times 64 \times 3$) → numpy 배열로 reshape → 5배 확대해 표시
# STABLE: <라벨> <conf> seq=N	"#"로 시작하는 사람용 로그 중 STABLE만 인식 → 확정 결과 박스로 강조 표시 (예: <code># STABLE: bottle 0.89 seq=3</code>)

5-2. 화면 구성

- **왼쪽:** 모델이 본 64×64 입력을 5배 확대(픽셀이 보이는 것이 의도). 시연 시 학생이 "모델은 이렇게 작은 그림으로 판단한다"는 사실을 직관적으로 깨닫는 장면입니다.
- **오른쪽 위:** 실시간 인식(매 추론) — st.metric으로 클래스·확신도·추론 시간 표시 + 프로그레스 바
- **오른쪽 아래:** 안정화 확정 결과 — 같은 답이 3회 연속이고 확신도 0.70 이상일 때만 갱신되는 초록 박스
- **아래쪽:** 최근 20회 인식 이력 표 (시각, 클래스, 확신도, 추론 ms)

5-3. 확장 — 학생 미션 ②에서 손볼 수 있는 곳

- **이력 표를 SQLite에 저장** — "오늘 가장 많이 인식된 물체"·"시간대별 분포" 같은 분석 페이지 추가. 기존 MQTT 프로젝트의 시리얼→SQLite 패턴 그대로 이식 가능.
- **색 보정 / 게이지 디자인** — st.metric 대신 plotly의 Indicator로 다이얼식 게이지 표현, 확신도에 따라 색 변화 등.

- **다국어 라벨 표시** — LBL 영문 라벨을 한글로 매핑하는 사전을 두면, 모델은 영문으로 학습되어 깔끔하고 화면에는 한글로 표시됩니다.

색이 이상하게 보일 때

펌웨어가 보내는 영상은 RGB 순서지만, 카메라 센서의 디버그 옵션에 따라 색 채널이 바뀔 수 있습니다. 영상이 푸르스름하거나 붉으스름하면 monitor.py의 `reshape` 부분에서 `[:, :, ::-1]` 을 추가해 BGR로 뒤집어 보세요.

제어 노드 — 무선 출력 (BLE 수신 → OLED 표시)

STEP 06

담당: Arduino Nano ESP32 + Grove 실드 + Grove OLED 1.12" V2 — BLE로 받은 결과를 화면에 글자로 표시

WHAT — 이 단계가 하는 일

BLE Central로서 비전 노드를 찾아 연결하고 notify를 구독합니다. 3바이트가 도착하면 클래스 번호를 라벨로 바꿔 OLED에 큰 글씨로 표시 — "병", "컵", "안경" 식으로요. PC가 없어도 보드 두 개와 OLED 하나만으로 시연 완성.

WHY — 왜 서보가 아니라 OLED인가

서보는 외부 5V 전원·강압 모듈·전원 합류 배선이 필요해 학생 차시에 부담이 큼니다. OLED는 Grove 케이블 한 줄로 끝나고(I2C 3.3V 직결), 결과를 글자로 직접 보여 줘 직관성도 더 좋습니다. 서보 출력은 **심화 미션**으로 별도 진행합니다.

6-1. 배선 — Grove 케이블 한 줄

장치	연결	전압
Grove OLED 1.12" V2	Grove 실드의 I2C 포트 중 하나에 케이블 1줄로 연결	3.3V (자동, 실드에서 공급)

왜 이렇게 간단한가

Grove 시스템은 "**4핀 케이블 하나로 전원·신호·접지 모두 통일**"되도록 설계했습니다. I2C 통신용 핀(SCL, SDA) + 3.3V + GND가 한 케이블에 모여 있어, 학생이 핀을 잘못 꽂을 가능성이 없습니다. 실드의 I2C 포트 어느 곳에 꽂든 동작합니다.

실드 전원 스위치는 3V3에 고정

Nano ESP32는 3.3V 로직 보드. 실드 VCC 스위치를 5V로 올리면 ① 5V 레일은 보드 뒷면 VUSB 점퍼 미납땜 상태에선 죽어있고, ② 이후 켜는 모든 Grove 센서가 5V 구동되어 3.3V 핀에 5V 신호가 들어가는 손상 경로가 열립니다. 스위치는 항상 3V3로 두세요.

6-2. 라이브러리 설치

1. Arduino IDE → 라이브러리 매니저 → 검색 "**U8g2**" → **U8g2 by oliver** 설치 (가장 보편적인 OLED 라이브러리. SH1107 외에도 수백 종의 디스플레이를 지원)
2. 같은 매니저에서 "**ESP32 BLE Arduino**"는 ESP32 보드 패키지에 기본 포함이라 별도 설치 불필요

6-3. 펌웨어

Grove OLED 1.12" V2(SH1107G 칩, 128×128 픽셀)는 U8g2의 `U8G2_SH1107_SEEED_128X128` 생성자로 동작합니다.

```
/* =====
 * Vision Gauge - 제어 노드 (Nano ESP32 + Grove OLED 1.12" V2)
 * -----
 * BLE Central로서 비전 노드에 연결해 인식 결과를 OLED에 표시.
 *
 * 핵심 동작:
 * 1) 비전 노드("VisionGauge") 탐색 -> 연결
 * 2) 라벨 특성(0x...003)을 read -> LABELS[] 동적 구성
 *   (비전 노드에서 학습한 라벨이 자동으로 따라옴 - 코드 수정 불필요)
 * 3) 결과 특성(0x...002) 구독 -> 3바이트 수신 시 OLED 갱신
 * 4) 상단에 BLE 연결/수신 상태 인디케이터 표시
 *
 * OLED: Grove 1.12" V2 (SH1107G, 128x128, I2C)
 * 라이브러리: U8g2 by oliver, ESP32 내장 BLE
 * ===== */
#include <BLEDevice.h>
#include <Wire.h>
#include <U8g2Lib.h>

// ----- Grove OLED 1.12" V2 (SH1107G, 128x128) -----
U8G2_SH1107_SEEED_128X128_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);

// ----- BLE UUID (비전 노드와 동일해야 함) -----
#define SERVICE_UUID "7e400001-b5a3-f393-e0a9-e50e24dcca9e"
#define CHAR_UUID "7e400002-b5a3-f393-e0a9-e50e24dcca9e" // 결과 (3바이트)
#define LABEL_UUID "7e400003-b5a3-f393-e0a9-e50e24dcca9e" // 라벨 문자열

// ----- 라벨 (BLE로 자동 수신 - 직접 입력 불필요) -----
#define MAX_LABELS 8
String LABELS[MAX_LABELS]; // 비전 노드에서 받은 라벨이 채워짐
int numLabels = 0;

// ----- 상태 변수 -----
volatile int curClass = -1; // 현재 표시 중인 클래스 인덱스
volatile int curConf = 0; // 확신도 (0~100)
```

```

volatile bool updated = false; // 새 데이터 도착 플래그
volatile uint32_t lastRxMs = 0; // 마지막 수신 시각 (수신 인디케이터용)
bool bleConnected = false;

BLEClient* pClient = nullptr;
BLERemoteCharacteristic* pResultChar = nullptr;

/* ----- 결과 특성 notify 콜백 -----
 * 비전 노드가 200ms마다 푸시하는 3바이트 [class_id, conf, seq] 수신 */
static void notifyCB(BLERemoteCharacteristic* c,
                    uint8_t* data, size_t len, bool isNotify) {
    if (len < 3) return;
    curClass = data[0];          // 클래스 인덱스 (LABELS[] 미수신이어도 저장)
    curConf = data[1];          // 확신도 0~100 정수
    updated = true;
    lastRxMs = millis();        // 수신 시각 기록 -> 인디케이터 점멸
}

/* ----- 비전 노드 연결 + 라벨 수신 ----- */
bool connectVisionNode() {
    BLEScan* scan = BLEDevice::getScan();
    scan->setActiveScan(true);

    // esp32 2.0.x: start()가 값을 반환 (3.x면 BLEScanResults* + res->)
    BLEScanResults res = scan->start(5);
    for (int i = 0; i < res.getCount(); i++) {
        BLEAdvertisedDevice d = res.getDevice(i);
        if (d.haveServiceUUID() && d.isAdvertisingService(BLEUUID(SERVICE_UUID))) {
            pClient = BLEDevice::createClient();
            if (!pClient->connect(&d)) return false;

            BLERemoteService* svc = pClient->getService(SERVICE_UUID);
            if (!svc) return false;

            // --- 라벨 특성 read -> LABELS[] 구성 ---
            BLERemoteCharacteristic* lblChar = svc->getCharacteristic(LABEL_UUID);
            if (lblChar && lblChar->canRead()) {
                // 비전 노드가 라벨을 기록하기 전에 읽힐 수 있어 최대 5회 재시도
                String csv = "";
                for (int retry = 0; retry < 5; retry++) {
                    csv = String(lblChar->readValue().c_str());
                    csv.trim();
                    if (csv.length() > 0) break; // 값을 받으면 종료
                    delay(300); // 비었으면 잠시 후 재시도
                }
                Serial.print("라벨 특성 read: ["); Serial.print(csv); Serial.println("]");

                if (csv.length() > 0) {
                    numLabels = 0;
                    int start = 0;
                    while (numLabels < MAX_LABELS) {
                        int comma = csv.indexOf(',', start);
                        if (comma < 0) {
                            LABELS[numLabels++] = csv.substring(start);
                            break;
                        }
                        LABELS[numLabels++] = csv.substring(start, comma);
                        start = comma + 1;
                    }
                    Serial.print("라벨 "); Serial.print(numLabels); Serial.println("개 파싱 완료");
                    for (int k = 0; k < numLabels; k++) {

```

```

        Serial.print(" "); Serial.print(k); Serial.print(" ");
        Serial.println(LABELS[k]);
    }
} else {
    Serial.println("경고: 라벨 특성이 비어 있음 - 비전 노드 step3 최신본인지 확인");
}
} else {
    Serial.println("경고: 라벨 특성(0x...003)을 못 찾음 - UUID 확인");
}

// --- 결과 특성 구독 ---
pResultChar = svc->getCharacteristic(CHAR_UUID);
if (!pResultChar || !pResultChar->canNotify()) return false;
pResultChar->registerForNotify(notifyCB);
return true;
}
}
return false;
}

/* ----- 상단 상태 바 그리기 -----
* 좌측: BLE 연결 아이콘 / 우측: 수신 점멸 표시 */
void drawStatusBar() {
    oled.setFont(u8g2_font_5x8_tr);

    // BLE 연결 상태 (좌상단)
    if (bleConnected) {
        oled.drawStr(2, 8, "BLE:ON");
        // 연결 아이콘 (채워진 원)
        oled.drawDisc(40, 5, 3);
    } else {
        oled.drawStr(2, 8, "BLE:--");
        oled.drawCircle(40, 5, 3); // 빈 원
    }

    // 수신 인디케이터 (우상단) - 최근 600ms 안에 수신했으면 채움
    bool rxActive = (millis() - lastRxMs) < 600;
    oled.drawStr(86, 8, "RX");
    if (rxActive) {
        oled.drawBox(104, 1, 8, 8); // 채워진 사각 (수신 중)
    } else {
        oled.drawFrame(104, 1, 8, 8); // 빈 사각 (대기)
    }

    oled.drawLine(0, 12, 128); // 구분선
}

/* ----- 메인 화면 그리기 ----- */
void drawScreen() {
    oled.clearBuffer();
    drawStatusBar();

    if (!bleConnected) {
        oled.setFont(u8g2_font_ncenB10_tr);
        oled.drawStr(8, 60, "connecting");
        oled.drawStr(8, 80, "...");
    } else if (curClass < 0) {
        oled.setFont(u8g2_font_ncenB10_tr);
        oled.drawStr(8, 60, "waiting");
        oled.drawStr(8, 80, "for data");
    } else {

```

```

// --- 인식된 라벨 ---
// LABELS[]가 비었거나 범위 밖이면 인덱스로 fallback (디버깅용)
String lab;
if (curClass >= 0 && curClass < numLabels && LABELS[curClass].length() > 0) {
    lab = LABELS[curClass];
} else {
    lab = "C" + String(curClass);    // 라벨 미수신 시 "C0","C1"... 로 표시
}

// 라벨 길이에 따라 폰트 자동 선택 (긴 라벨도 안 잘림)
if (lab.length() <= 6)        oled.setFont(u8g2_font_ncenB18_tr);
else if (lab.length() <= 9)   oled.setFont(u8g2_font_ncenB14_tr);
else                          oled.setFont(u8g2_font_ncenB10_tr);
oled.drawStr(6, 50, lab.c_str());

// --- 확신도 숫자 ---
char buf[24];
snprintf(buf, sizeof(buf), "conf %d%%", curConf);
oled.setFont(u8g2_font_6x12_tr);
oled.drawStr(6, 78, buf);

// --- 확신도 bar 게이지 ---
int barX = 6, barY = 88, barW = 116, barH = 16;
oled.drawFrame(barX, barY, barW, barH);          // 외곽
int fillW = (curConf * (barW - 4)) / 100;         // 채움 길이
oled.drawBox(barX + 2, barY + 2, fillW, barH - 4); // 내부 채움

// --- 눈금 (25/50/75%) ---
for (int p = 25; p < 100; p += 25) {
    int tx = barX + 2 + (p * (barW - 4)) / 100;
    oled.drawVLine(tx, barY + barH + 2, 3);
}
oled.setFont(u8g2_font_4x6_tr);
oled.drawStr(barX, barY + barH + 12, "0");
oled.drawStr(barX + barW - 16, barY + barH + 12, "100");
}

oled.sendBuffer();
}

void setup() {
    Serial.begin(115200);
    delay(500);          // 시리얼 안정화 대기
    Serial.println();
    Serial.println("=== 제어 노드 부팅 시작 ===");

    Wire.begin();
    Serial.println("1) I2C 초기화 완료");

    oled.begin();
    Serial.println("2) OLED 초기화 완료");
    drawScreen();        // "connecting..." 표시

    BLEDevice::init("");
    Serial.println("3) BLE 초기화 완료 - 비전 노드 탐색 시작");

    int tries = 0;
    while (!connectVisionNode()) {
        tries++;
        Serial.print("비전 노드 재탐색... ("); Serial.print(tries); Serial.println("회)");
        delay(1000);
    }
}

```



```

}
bleConnected = true;
Serial.println("4) 비전 노드 연결 성공!");
drawScreen();          // "waiting for data" 표시
}

void loop() {
  // 새 데이터가 왔거나, 수신 인디케이터 점멸을 위해 주기적으로 다시 그림
  static uint32_t lastDraw = 0;
  if (updated || (millis() - lastDraw > 200)) {
    updated = false;
    lastDraw = millis();
    drawScreen();
  }
  delay(20);
}

```

U8g2 라이브러리 핵심 사용법

`clearBuffer()` → `drawStr()` · `setFont()` 로 그리고 → `sendBuffer()` 로 한 번에 출력. 매번 화면 전체를 다시 그리는 방식이라 깜빡임이 없습니다. 폰트는 `u8g2_font_ncenB18_tr` (18px Bold) 같은 식으로 크기·굵기 선택 가능 — U8g2 공식 문서에 수백 종이 있습니다.

라벨이 자동으로 따라옵니다 — 코드 수정 불필요

이 제어 노드 코드에는 `LABELS[]` 를 직접 적지 않습니다. 대신 **연결 직후 비전 노드로부터 라벨 목록을 BLE로 받아**(라벨 특성 0x...003) 자동으로 채웁니다. 즉 비전 노드에서 학습한 라벨이 무엇이든, 제어 노드는 그대로 표시합니다. 모둠이 `bottle, marker` 로 학습하면 OLED에도 그렇게 뜹니다 — ESP32 코드를 다시 업로드할 필요가 없습니다.

"Label 1, Label 2"로 떴다면?

OLED에 실제 라벨이 아니라 `Label 1` 같은 게 뜬다면 ① 비전 노드 펌웨어가 **라벨 특성을 보내는 최신 버전(step3)**인지, ② 연결 직후 시리얼 모니터에 "라벨 수신: ..." 줄이 출력되는지 확인하세요. 비전 노드가 구버전이면 라벨 특성이 없어 제어 노드가 인덱스만 표시합니다.

6-4. 동작 확인

1. 비전 노드 펌웨어(STEP 4-C BLE 송신 추가본) 업로드 후 동작 중
2. 제어 노드(이 코드) 업로드 → 보드 부팅 → OLED에 "Vision Gauge / connecting..." 표시
3. 5~10초 안에 "waiting for data"로 전환되면 BLE 연결 성공
4. 비전 노드 앞에 학습한 물체를 들이대면 OLED에 **해당 라벨 + 확신도**가 표시됨 (예: 모둠이 `bottle` 클 래스로 학습했다면 물병을 들이댔을 때 "bottle / conf 91%" 표시)
5. OLED 상단의 **상태 바** 확인: 좌측 `BLE:ON` + 채워진 원 = 연결됨, 우측 `RX` + 채워진 사각 = 방금 데이터 수신. 물체를 바꿀 때마다 RX 표시가 깜빡이면 송수신 정상.

6. 화면 하단의 **막대 게이지**가 확신도(0~100%)를 길이로 표시 — 25/50/75% 눈금으로 직관적으로 읽힘.
7. 물체를 바꾸면 OLED 표시도 0.6초 이내(안정화 지연) 따라 변경

6-5. 심화 미션 — 서보 액추에이터로 확장 (외부 전원 필요)

OLED는 결과를 "글자로" 보여주지만, 서보는 "물리적 움직임으로" 보여줍니다 — 인식된 클래스에 따라 게이지 바늘이 다른 각도를 가리키는 형태. 외부 5V 전원이 필요해 학생 차시 공통 과제에서는 제외했지만, 의지가 있는 모둠은 다음 추가 부품으로 도전 가능합니다:

- SG90 서보 모터 1개
- LM2596 강압 모듈 (12V → 5V 가변 출력)
- 9~12V DC 어댑터 1개
- Grove to male/female 점퍼 또는 미니 브레드보드

배선·코드·운영 팁은 **교사용 매뉴얼의 심화 부록**을 참조하세요. 두 출력(OLED + 서보)을 동시에 동작시키는 것도 가능하며 — 같은 BLE 콜백에서 OLED 갱신과 서보 각도 갱신을 같이 처리하면 됩니다.

STEP 07 (심화)

서보 액추에이터 + 게이지 다이얼 제작

외부 5V 전원·강압 모듈이 필요한 확장 과제 — 모둠 선택형

WHAT — 이 단계가 하는 일

OLED 화면 출력에 더해 **서보 모터로 바늘을 움직여** 게이지처럼 인식 결과를 물리적으로 표시합니다. 동일한 BLE 데이터를 OLED와 서보가 동시에 받아 두 가지 방식으로 표현.

WHY — 왜 심화로 빠졌나

SG90 서보는 3.3V 로직 보드로 직접 구동이 안 됩니다(전류·전압 부족 → 보드 리셋). 외부 5V 전원, LM2596 강압 모듈, 공통 접지 배선이 필요해 학생 차시에서 다 같이 진행하기엔 부담이 큼니다.

준비물 (추가)

- SG90 서보 모터 1개
- LM2596 강압 모듈 (9~12V → 5V 가변)
- 9~12V DC 어댑터 1개 (보드와 같이 쓰면 안 됨 — 분리 전원)
- Grove-to-male/female 점퍼 또는 미니 브레드보드 + WAGO 커넥터

- 다이얼 공작 재료: A4 용지·자·각도기·빨대 또는 두꺼운 종이

핵심 원리 — 한 줄 요약

서보 외부 전원 — 3원칙

- ① **분리 급전** — 서보 VCC는 ESP32가 아닌 외부 강압 모듈에서 (직결 시 보드 brown-out 리셋).
- ② **공통 GND** — 외부 전원의 GND와 ESP32의 GND를 한 점에서 합류시켜야 PWM 신호의 기준이 맞음.
- ③ **무부하 전압 확인 후 연결** — LM2596 트리머는 멀티턴이라 이전 사용자의 설정이 남아 있음. 서보 미연결 상태로 5.0V 확인 후에야 서보 연결.

코드 통합

STEP 06의 OLED 코드에 서보 라이브러리 + `notifyCB` 안에 한 줄만 추가하면 됩니다:

```
// 추가 #include
#include <ESP32Servo.h>

// 추가 전역
Servo gauge;
const int ANGLES[] = { 0, 90, 180 }; // LABELS와 같은 순서: 첫 클래스 0°, 둘째 90°, 셋째 180°

// notifyCB 안에 한 줄 추가:
gauge.write(ANGLES[cls]);

// setup() 안에 한 줄 추가 (Grove D6 신호선):
gauge.attach(6);
gauge.write(ANGLES[2]); // 시작 위치는 unknown (중앙 180°)
```

자세한 배선 다이어그램, 다이얼 공작 가이드, 부드러운 움직임(easing) 구현은 **교사용 매뉴얼** 부록을 참조하세요.

통합 테스트 절차

STEP 08

아래 순서대로 하나씩 확인합니다. **한 번에 전부 켜고 "왜 안 되지"를 찾는 것이 최악의 디버깅입니다** — 각 단계는 앞 단계가 통과한 뒤에만 의미가 있습니다.

- ☐ ① **모델 단독** — Edge Impulse Model testing에서 테스트 정확도 85%+ 확인

☐ ② 비전 노드 추론 (STEP 4-A) — vision_node_step1_test.ino로 시리얼 모니터 RES/STABLE 줄 확인

☐ ③ USB 스트리밍 + monitor.py (STEP 4-B) — vision_node_step2_usb.ino + monitor.py로 영상·게이지 동작 확인

☐ ④ BLE 송신 검증 — 비전 노드에 BLE 송신 코드 추가 후, 폰 nRF Connect로 "VisionGauge" 연결 → 캐릭터리스틱 구독 → 물체 교체 시 3바이트 갱신 확인

☐ ⑤ 제어 노드 OLED 표시 — Nano ESP32 + Grove OLED 1.12" V2에 STEP 06 코드 업로드. 부팅 시 "connecting" → "waiting for data" → 물체 인식 시 라벨·확신도 표시

☐ ⑥ 라벨 정합성 — 학습한 각 클래스 물체를 차례로 비춰, OLED에 표시되는 라벨이 일치하는지 확인. 어긋나면 코드의 LABELS[] 순서를 TI 라벨 알파벳순과 정확히 맞춰 재업로드

☐ ⑦ 내구 테스트 — 10분간 물체를 바꿔가며 방치, 끊김·리셋·표시 멈춤 없는지 관찰

☐ ⑧ (심화) 서보 추가 — STEP 07을 진행한 모뎀은 외부 5V 무부하 확인 → 서보 연결 → 각도 회전 확인

STEP 09

트러블슈팅 & 확장 과제

적용	증상	원인	해결
공통	Nicla 업로드 실패 / 포트 안 보임	부트로더 진입 필요	리셋 버튼 빠르게 2회(녹색 LED 점멸) 후 업로드
공통	BLE 코드 컴파일 오류	보드별 BLE 라이브러리 혼용	Nicla=ArduinoBLE, Nano ESP32=BLEDevice.h — 호환 불가
공통	비전 노드가 연결 직후 끊김	BLE.poll() 누락	loop()와 추론 루프 사이사이에 BLE.poll() 호출
공통	모델 빌드/업로드 시 메모리 부족	모델이 Nicla RAM 초과	Deployment에서 Quantized(int8), 입력 64×64 유지
모니터	대시보드가 포트를 못 엮	IDE 시리얼 모니터와 포트 충돌	둘 중 하나만 — IDE 모니터 닫고 streamlit 실행
모니터	영상이 안 뜸 / 간헐적으로 깨짐	base64 줄 잘림·길이 불일치	길이 검증(len==W*H*2)이 깨진 줄을 걸러줌 — 지속되면 보드 리셋
모니터	영상 색이 이상함 (보라/초록 틴트)	RGB565 바이트 오더	monitor.py의 dtype을 '>u2' ↔ '<u2' 교체
모니터	영상 fps가 낮음 (~5fps)	추론+전송 직렬 — 정상 사양	고fps가 필요하면 프레임을 N회에 1번만 송신하는 식의 역설계가 아니라, 이 수치가 설계값임을 안내
OLED	화면이 까만 채로 안 켜짐	Grove 케이블 헐거움 또는 I2C 포트가 아닌 곳에 꽂힘	실드의 I2C 라벨 포트에 케이블 재연결, 케이블 양쪽 단단히 결합
제어/컴파일	cannot convert 'BLEScanResults' to 'BLEScanResults*'	esp32 보드 버전 차이 (2.0.x는 값 반환)	코드 주석대로 <code>BLEScanResults res = scan->start(5);</code> (별 없음) + <code>res.</code> 사용
OLED	화면 켜지지만 깨진 패턴	U8g2 생성자 불일치 (V2가 아닌 다른 모델용)	<code>U8G2_SH1107_SEEED_128X128_F_HW_I2C</code> 정확히 사용 — V1은 SSD1327, V3은 다른 생성자
OLED	BLE 연결은 됐는데 화면이 안 바뀜	updated 플래그가 갱신 안 됨 / notifyCB 미등록	시리얼 모니터로 notifyCB 호출 확인 — 안 찍히면 BLE 구독 실패

OLED	한글 라벨이 깨짐	U8g2 기본 폰트는 영문만	라벨을 영문으로 유지하거나, <code>u8g2_font_unifont_t_korean1</code> 같은 한글 지원 폰트 사용 (플래시 사용량 ↑)
공통	라벨이 엉뚱하게 표시	LABELS[] 순서 불일치	EI Studio의 라벨 알파벳순과 코드 배열 순서 일치 확인
심화	서보 동작 시 보드 리셋	외부 전원 미사용 / 공통 GND 누락 / 전압 부족	STEP 07 부록 참조 — 분리 급전·공통 GND·무부하 5.0V 확인 3원칙
공통	실물 인식 정확도가 낮음	학습-시연 환경 차이	실제 거리·배경·조명으로 데이터 재수집 (Nicla 카메라 직접 수집 권장)
공통	unknown만 계속 인식	물체가 프레임에서 작음	카메라-물체 거리 단축, 학습 사진 거리와 일치 — 코어측 모니터 영상으로 직접 확인 가능

확장 과제 (학생 도전용)

- **OLED 게이지 그래픽** — 글자 외에 막대 그래프·아이콘으로 확신도 시각화 (U8g2의 `drawBox`·`drawCircle` 활용)
- **탐색 모드** — unknown 5초 지속 시 화면에 "찾는 중..." 애니메이션 표시
- **인식 통계** — `monitor.py` 이력을 SQLite에 적재, "오늘 가장 많이 인식된 물체" 대시보드
- **다중 액추에이터** — 클래스별 LED 색·부저 음 추가, Nicla 내장 RGB LED로 비전 노드 상태 표시
- **역방향 통신** — 모니터/웹 → BLE write → 비전 노드의 임계값 원격 조정
- **응용 전환** — 코어는 그대로 두고 분류 대상·액추에이터만 교체: 분리수거 분류 안내기, 재실 감지 도어 락, 부품 검수 합불 표시기 등